

Transformer

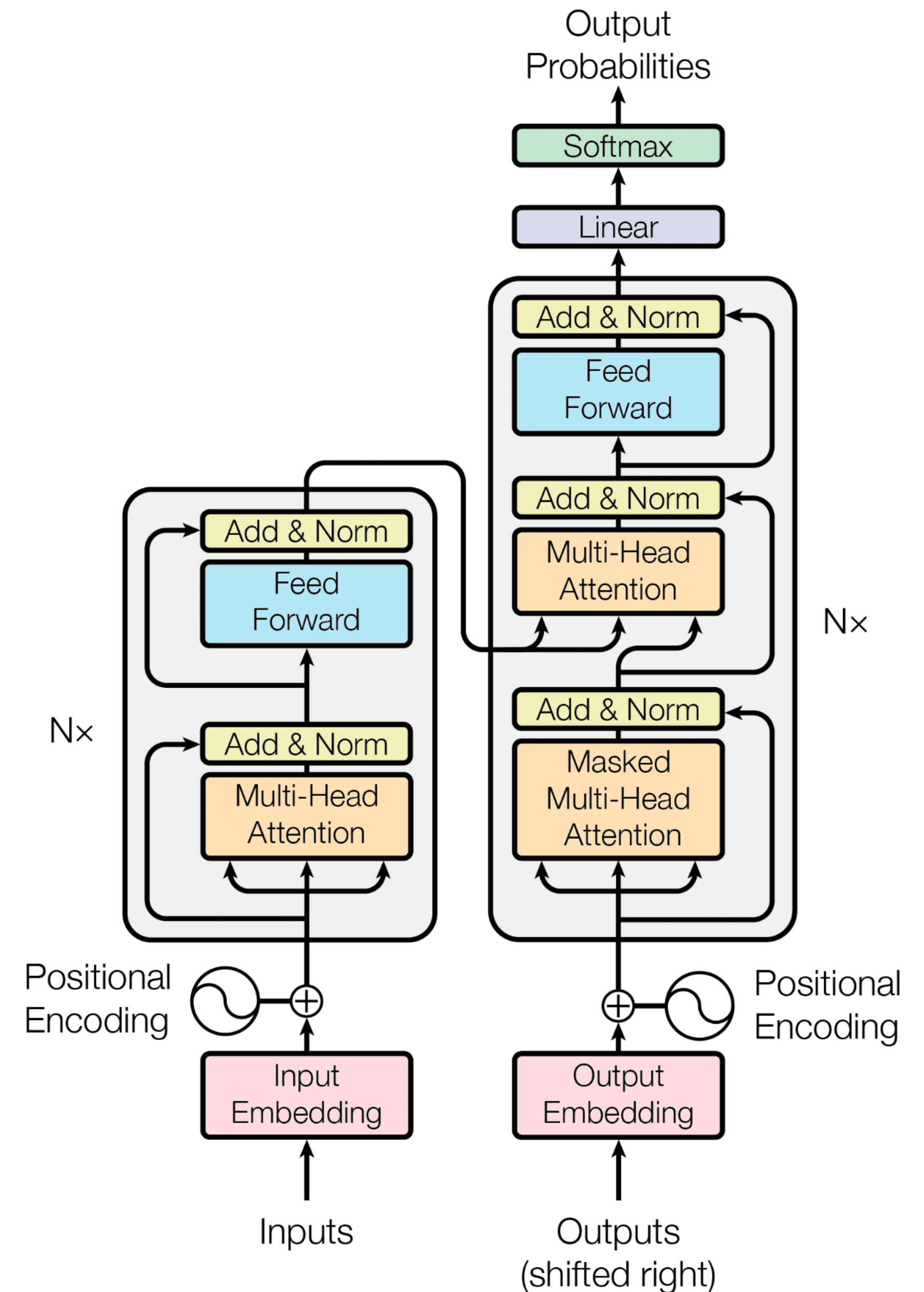
Attention is all you need

- Самая популярная архитектура с 2017 года
- ~200k цитат у оригинальной статьи

Разберемся с тем, как она работает

Компоненты:

1. Multi-Head Self-attention
2. Feed Forward Network
3. Masked Multi-Head Self-attention
4. Positional Encodings



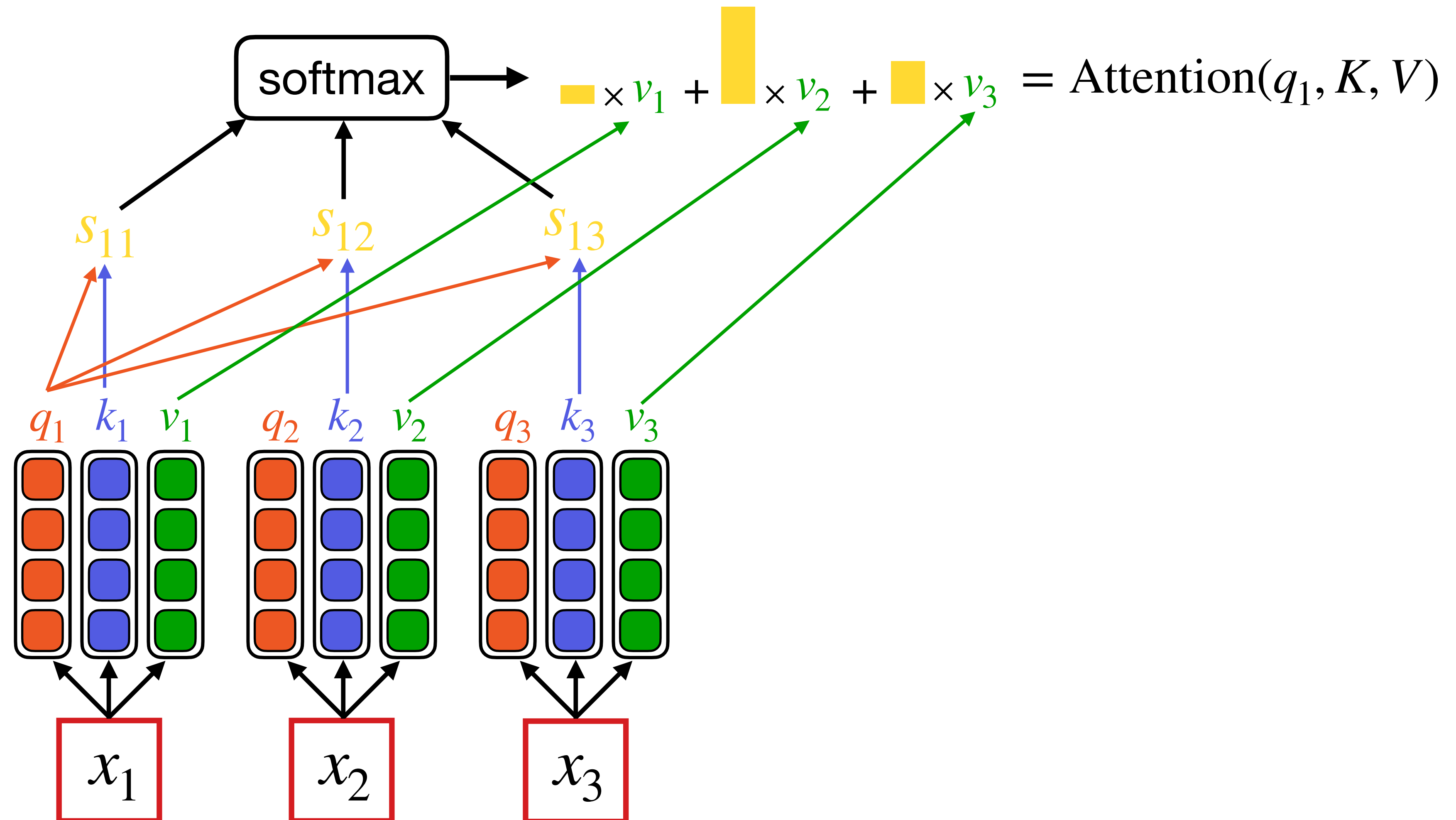
Self-attention

Позволяет каждому токенту посмотреть на все остальные.

q – запрос (query)

k – ключ (key)

v – значение (value)



Self-attention

Позволяет каждому токенту посмотреть на все остальные.

Query: $Q = W_q x + b_q$

Key: $K = W_k x + b_k$

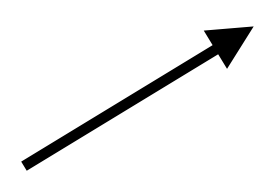
Value: $V = W_v x + b_v$

$$x \in \mathbb{R}^{n \times d}$$

$$W_q, W_k, W_v \in \mathbb{R}^{d \times d}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

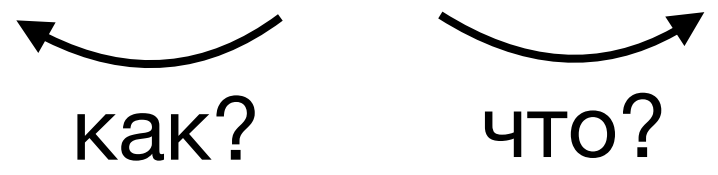
нормировочная
константа



Multi-Head Self-attention

Self-attention позволяет запрашивать только информацию одного вида.
А что если мы хотим узнать разные аспекты?

На улице ярко светит солнце

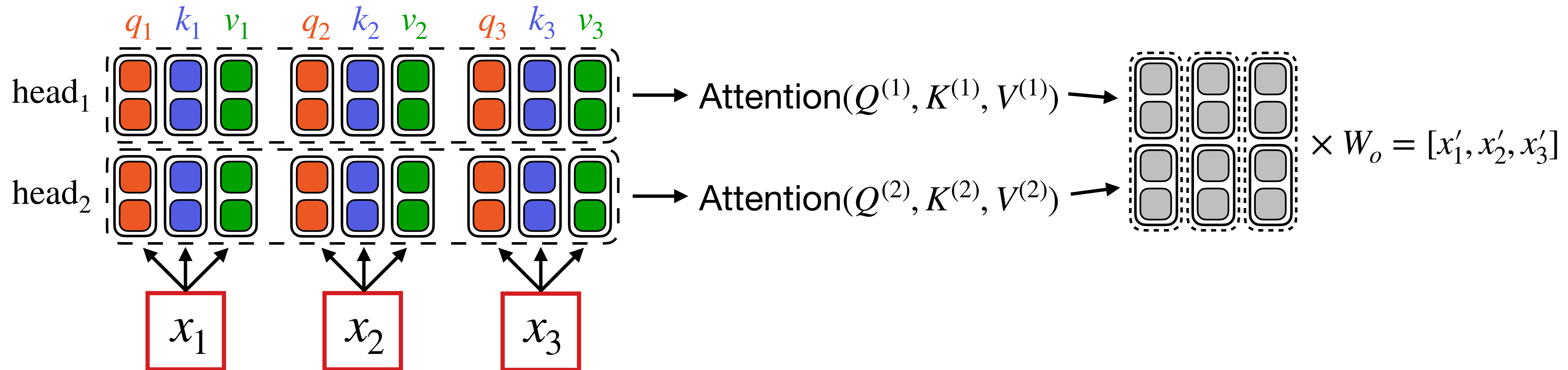


как? что?

Разделим внимание на несколько голов

Multi-Head Self-attention

- Делим каждый вектор q , k , v на равные части
- Считаем self-attention для каждой части отдельно
- Конкатенируем результат
- Домножаем на выходную матрицу



Multi-Head Self-attention

Query: $Q^{(i)} = W_q^{(i)}x + b_q^{(i)}$

Key: $K^{(i)} = W_k^{(i)}x + b_k^{(i)}$

Value: $V^{(i)} = W_v^{(i)}x + b_v^{(i)}$

$$\text{head}^{(i)} = \text{Attention}(Q^{(i)}, K^{(i)}, V^{(i)})$$

$$\text{MultiHead}(Q, K, V) = W_o \times [\text{head}^{(1)}, \dots, \text{head}^{(h)}]$$

$$W_q^{(i)}, W_k^{(i)}, W_v^{(i)} \in \mathbb{R}^{d_{\text{head}} \times d}$$

$$W_o \in \mathbb{R}^{d \times h \cdot d_{\text{head}}}$$

Self-attention не учитывает позиции

При перестановке токенов местами выход не изменится!

$$w = \text{softmax} \left(\frac{q_i K^T}{\sqrt{d}} \right)$$

$$\text{Attention}(q_i, K, V) = w_1 V_1 + \dots + w_n V_n$$

=> Не сможем различить два случая

очень хорошо, совсем **не** плохо

vs

не очень хорошо, совсем плохо

Positional Encoding

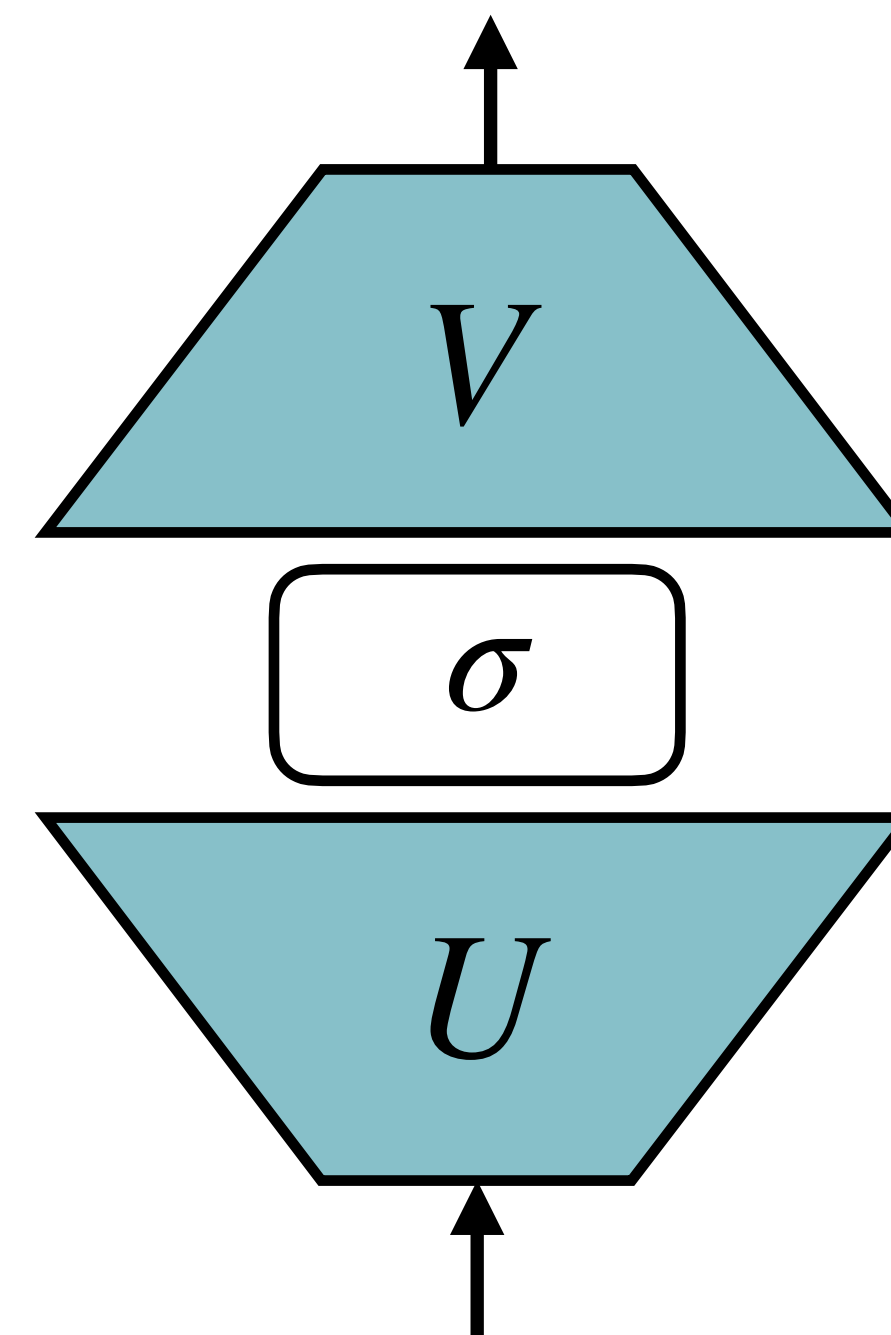
Необходимы для учета позиций токенов в тексте.

$$\begin{matrix} \begin{matrix} \boxed{} \\ \boxed{} \\ \boxed{} \\ \boxed{} \end{matrix} & = & \begin{matrix} \boxed{} \\ \boxed{} \\ \boxed{} \\ \boxed{} \end{matrix} & + & \begin{matrix} \boxed{} \\ \boxed{} \\ \boxed{} \\ \boxed{} \end{matrix} \\ x_i & & Emb(w_i) & & Emb_{pos}(i) \end{matrix}$$

Эмбеддинги позиций учатся вместе с остальными параметрами модели.

Feed Forward Network

- Полносвязная сеть из двух слоев.
- Первый слой увеличивает размерность, второй – возвращает обратно
- Используется для обработки информации, полученной после self-attention



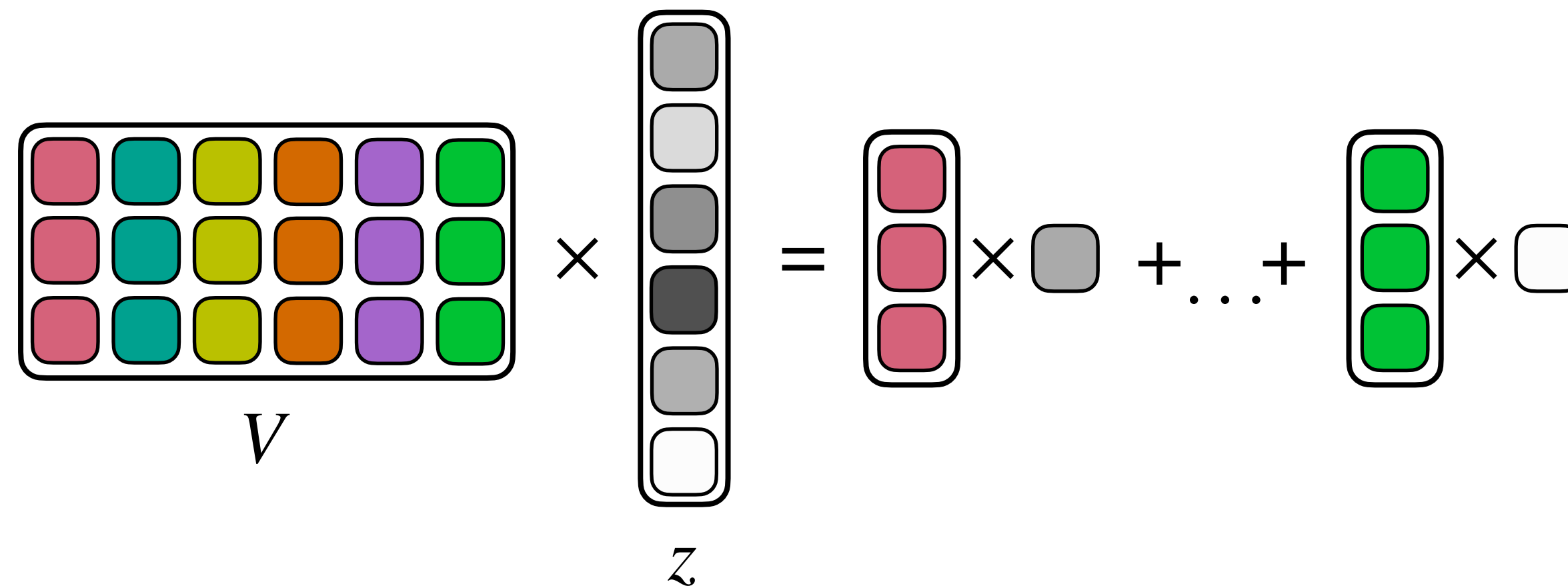
$$U \in \mathbb{R}^{2d \times d}$$

$$V \in \mathbb{R}^{d \times 2d}$$

FFN: Как ещё можно думать?

- FFN играет роль базы данных!
- U содержит ключи, а V – значения

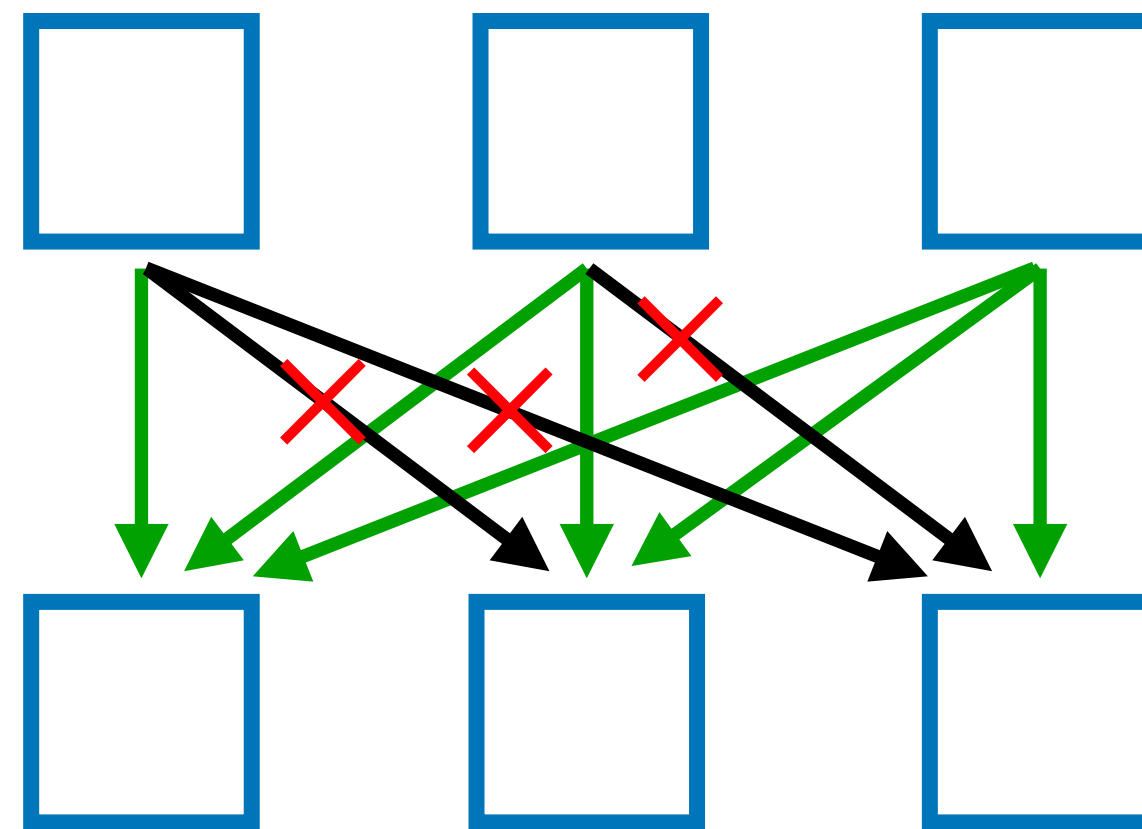
$$\sigma(Ux_i) = z \in \mathbb{R}^{2d}$$



- Столбцы V – определенные факты
- Мы взвешиваем эти факты согласно ключу z

Masked Self-attention

- При обучении генерации мы должны запретить декодеру смотреть на токены, идущие после текущего
- Декодер учитывает другие токены только в слое **self-attention**
- => Необходимо его модифицировать



Каждый охотник желает

Masked Self-attention

- Добавляем маску в подсчет внимания
- Маска явно запрещает смотреть вперед

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V$$

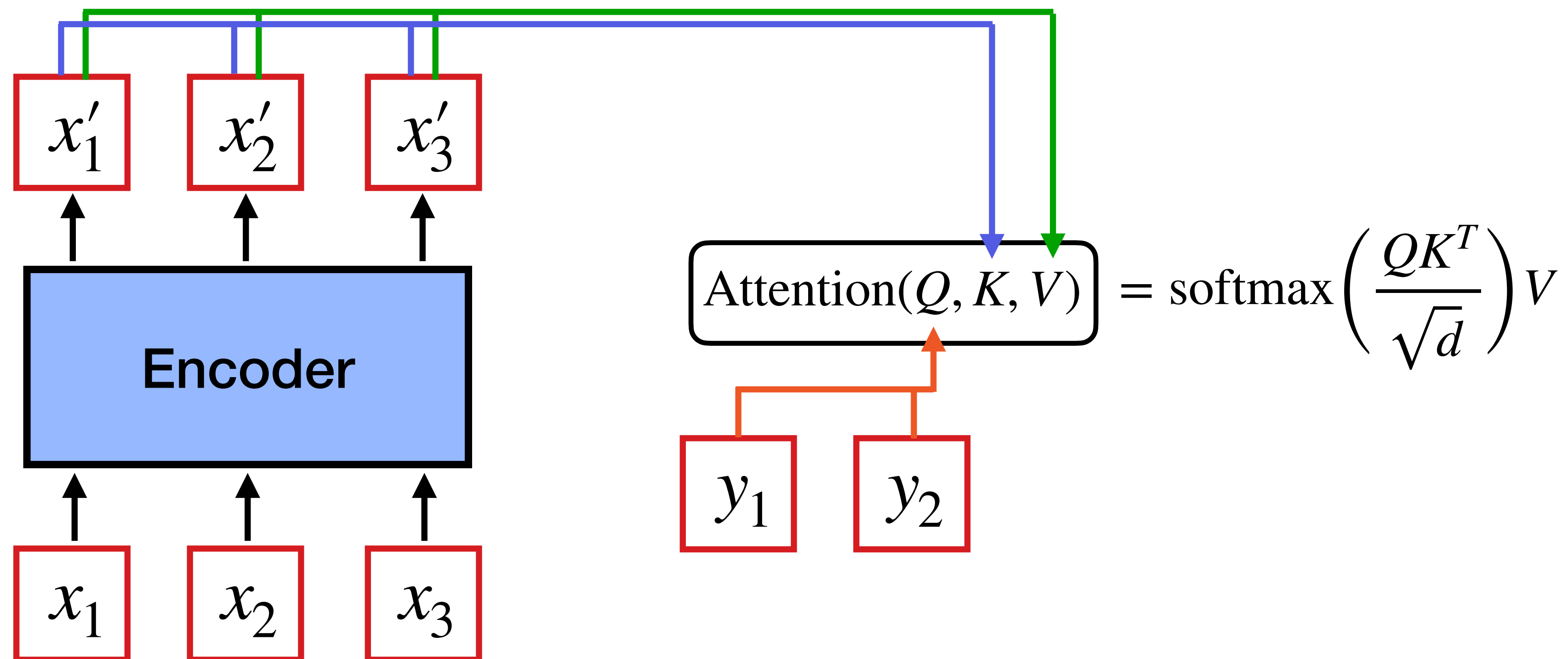
$$M_{ij} = \begin{cases} 0, & i \leq j \\ -\infty, & i > j \end{cases}$$

$M =$

0	$-\infty$	$-\infty$	$-\infty$
0	0	$-\infty$	$-\infty$
0	0	0	$-\infty$
0	0	0	0

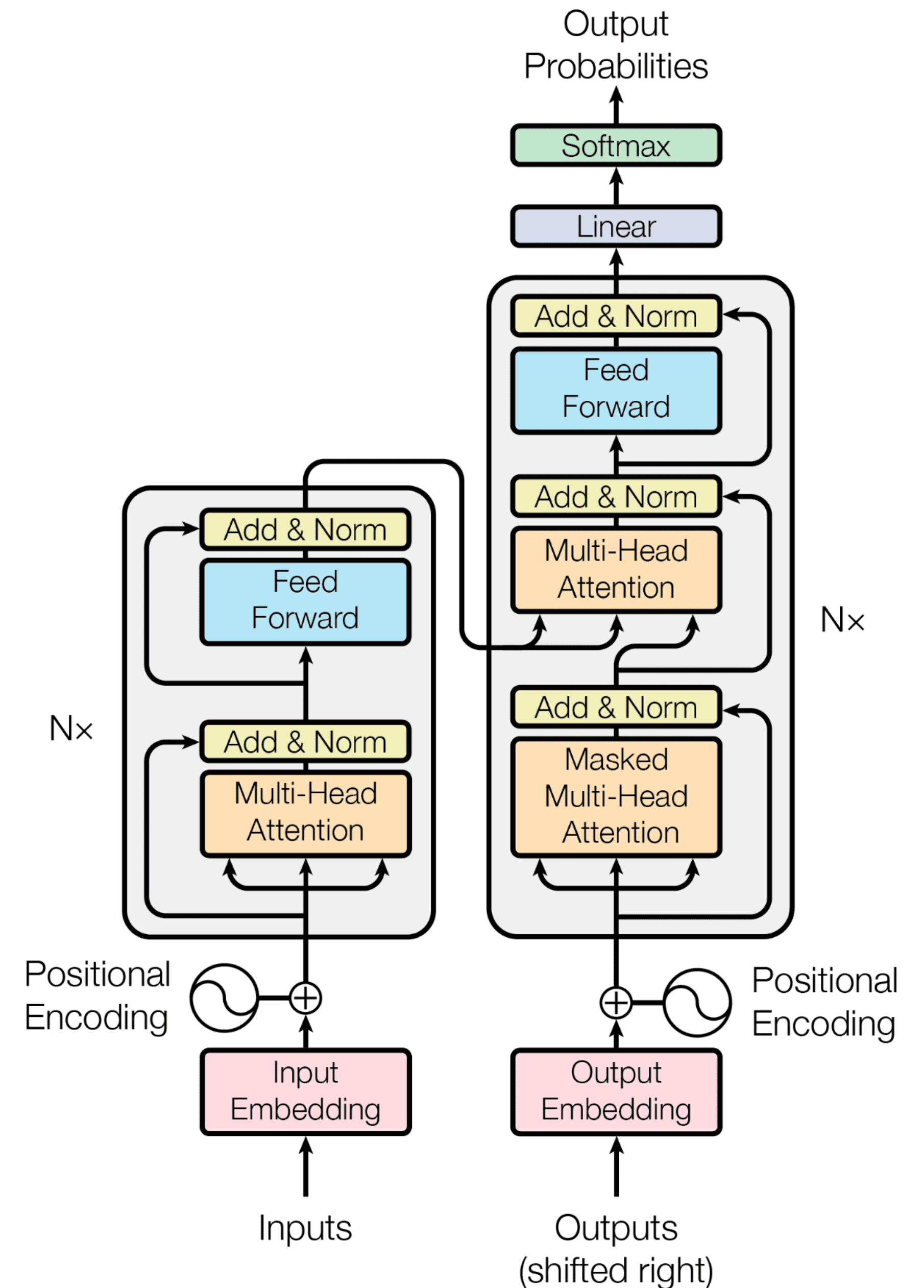
Cross Attention

- Используется для передачи информации от Encoder в Decoder
- Decoder "запрашивает" информацию, передавая **query** в attention
- Encoder передает **key** и **value**



Итого

- Трансформер создан специально для задач Seq2seq
- Блок **Encoder** состоит из:
 - Multi-Head Attention
 - Feed Forward Network
- После каждого слоя идет skip-connection и нормализация
- Блок **Decoder** использует
 - **Masked** Multi-Head Attention, чтобы не смотреть в будущее
 - **Cross-Attention**, чтобы брать информацию из Encoder
 - На выходе декодер предсказывает следующий токен

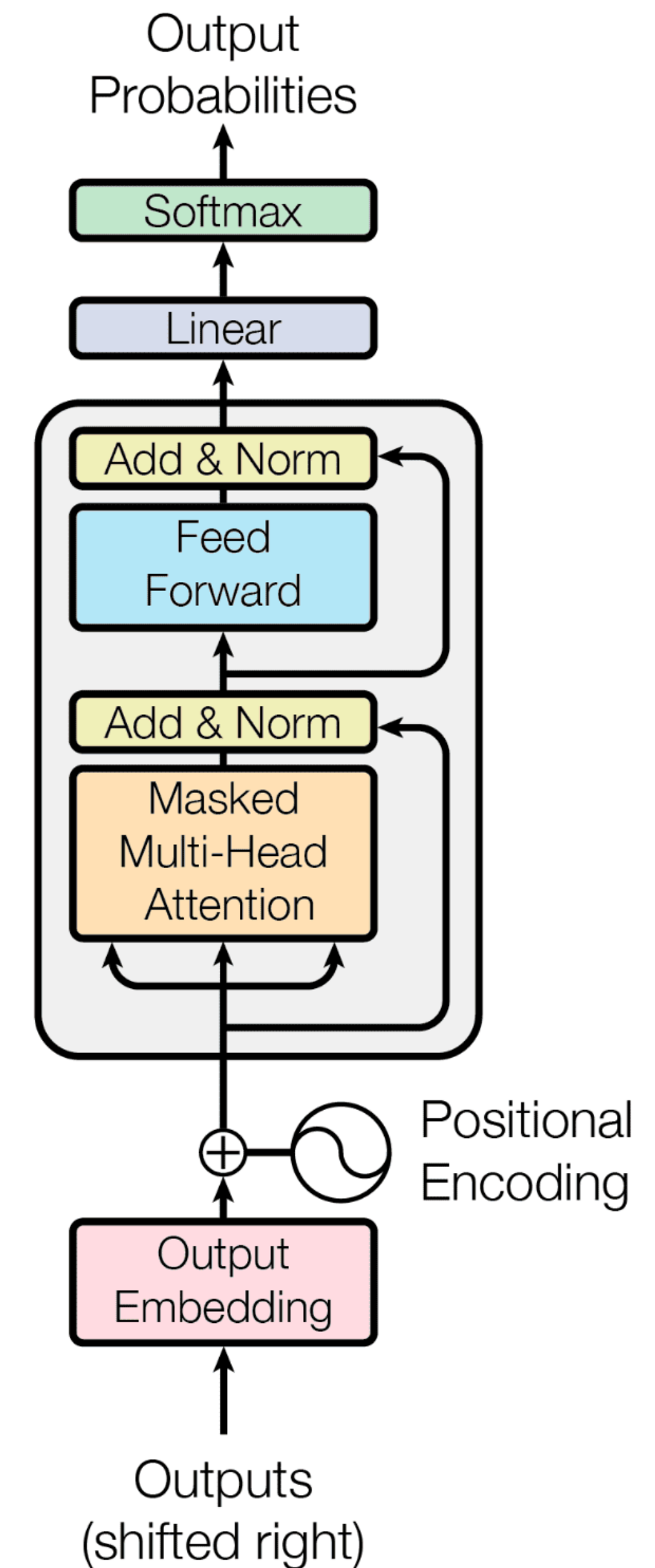


Что вообще поменялось в
ЭльЭльЭм с 2017 года?

GPT (2018)

Generative Pre-Training

- Вторая модель на основе Трансформера, способная предобучаться
- Применяется для **генерации** текста
- Использует **декодер** Трансформера без Cross Attention



GPT: Обучение

- GPT учится **языковому моделированию** (безусловная генерация текста)
- Благодаря способности Трансформера масштабироваться, GPT запоминает очень много информации в процессе обучения
- Любую задачу NLP можно свести к **генерации текста**
Классификация – генерация индекса класса
- Из-за этого GPT отлично дообучается на частные задачи

	Параметры	Данные
GPT-1	117M	5 гб
GPT-2	1.5B	40 гб
GPT-3	175B	45 тб

Attention

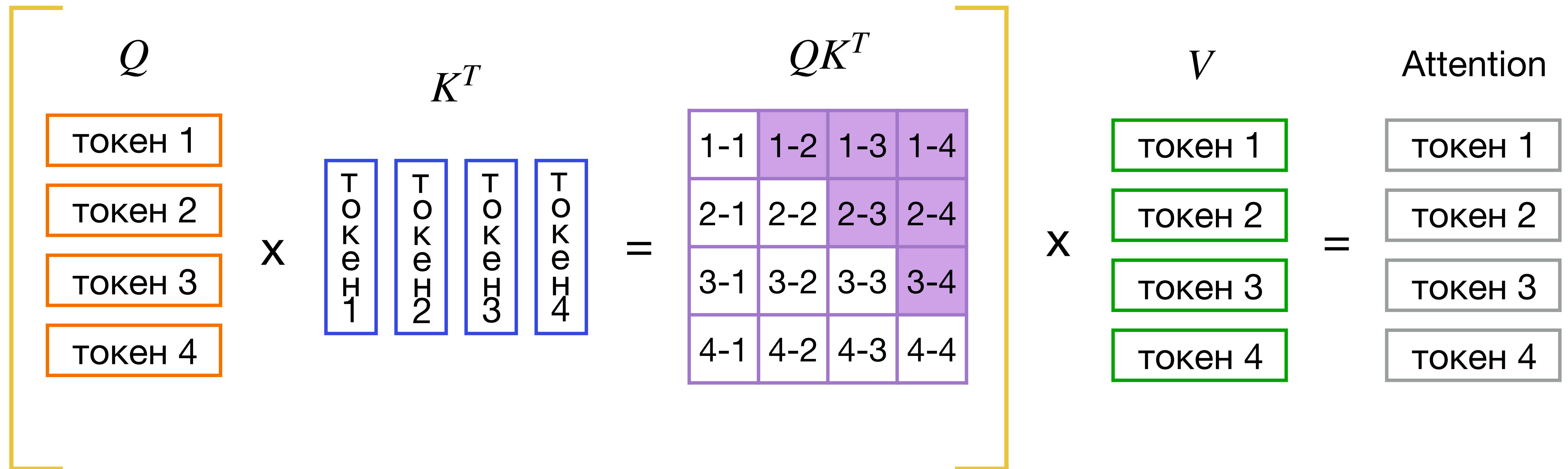
Генерация текста

- Все авторегрессионные модели генерируют текст по одному токену
- На каждой итерации необходимо пропускать всю последовательность через модель
- Один подсчет внимания занимает $O(l^2hd + lh^2d^2)$ операций

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Генерация текста

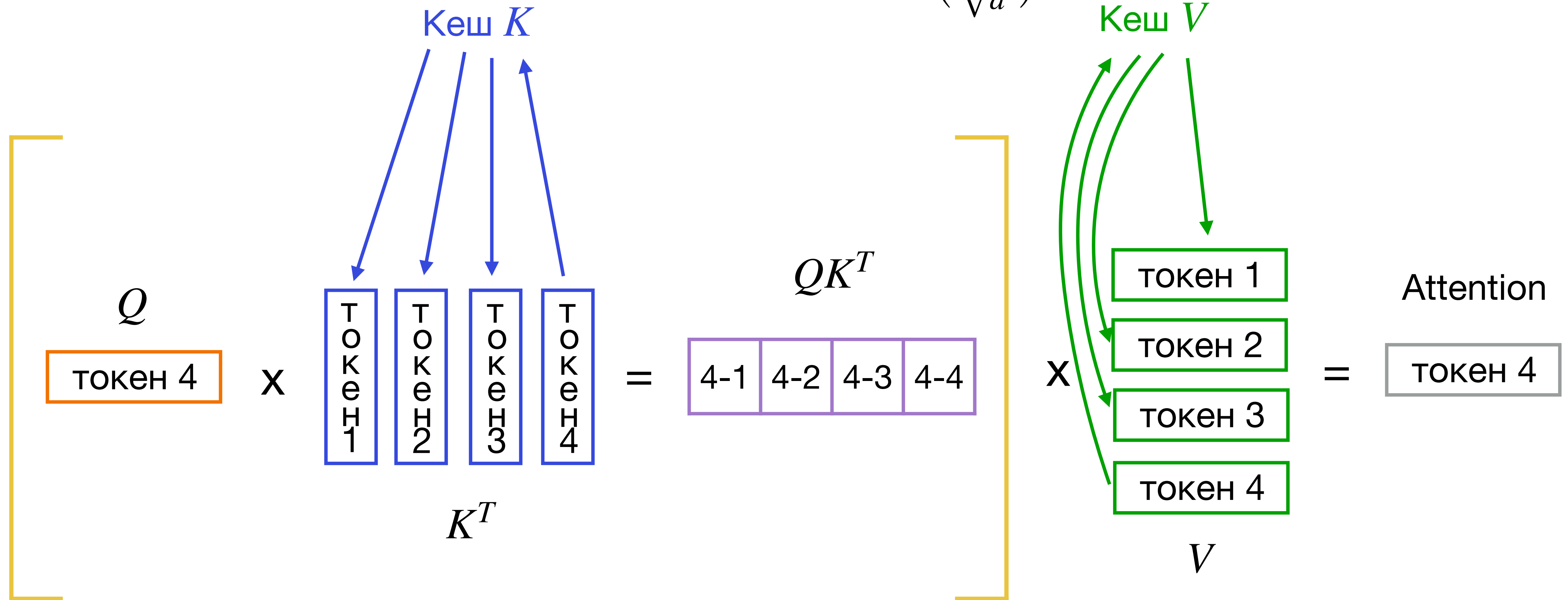
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$



Мы пересчитываем заново значения Q, K и V, хотя они не изменяются!
Каждый token зависит только от тех, что перед ним

KV кеширование

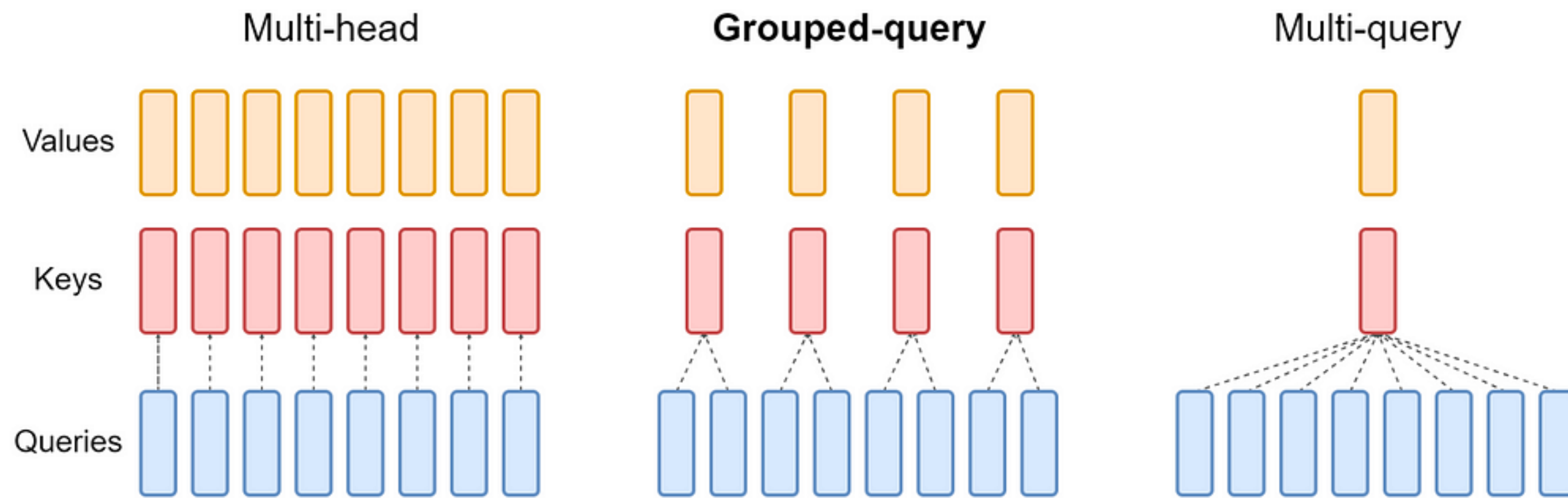
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$



На каждую итерацию тратим $O(lhd + hd^2)$ времени вместо $O(l^2hd + lh^2d^2)$!

Multi-Query Attention

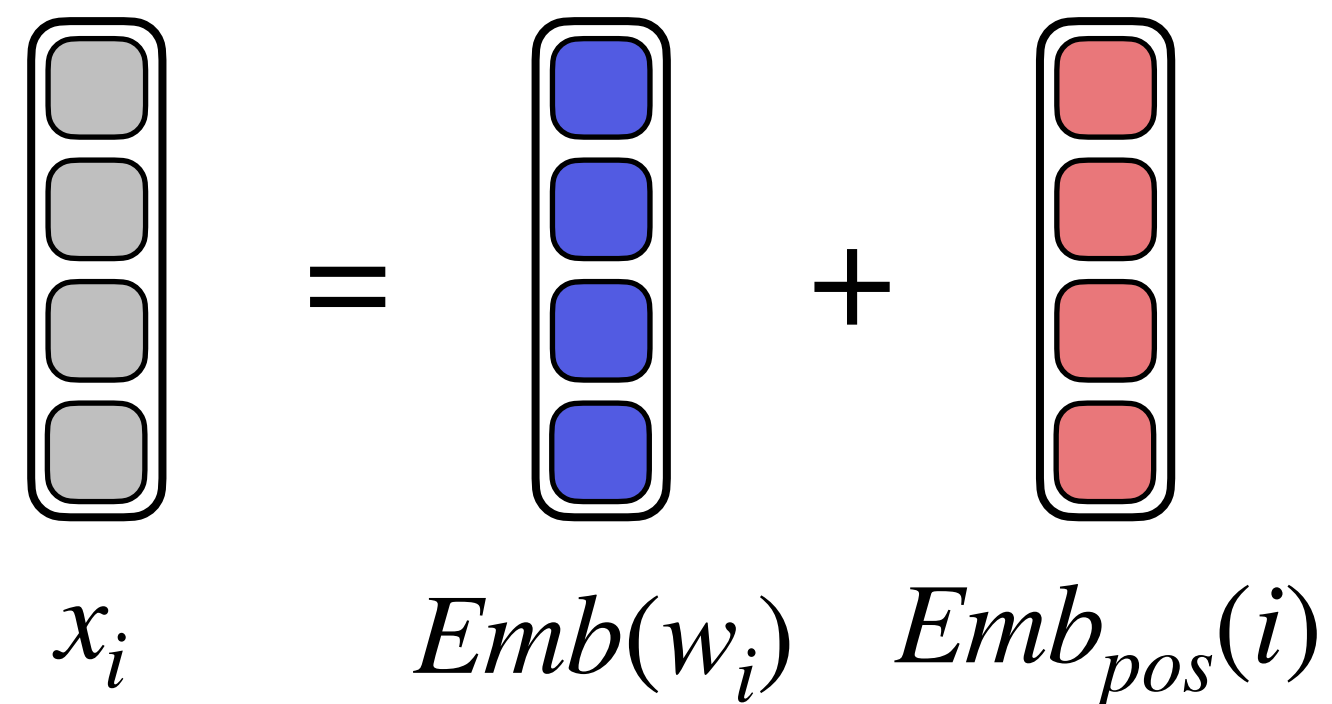
- KV кеширование требует хранения выходов всех слоев
- Скорость генерации падает из-за работы с памятью
- Можно оставить одну голову для K и V
- Обычно (Llama2, Falcon, Mistral, PaLM) выбирают средний вариант и оставляют около 8 голов



Позиционные энкодинги

Абсолютные позиционные энкодинги

- Не учитывается относительное расстояние
- Модель нельзя применять к более длинным текстам
- Важно не потерять информацию о позиции при работе с эмбедами



The diagram illustrates the formula for absolute positional encodings. It shows a vertical column of four gray rounded rectangles representing the input vector x_i . This is followed by an equals sign, then a vertical column of four blue rounded rectangles representing the word embedding $Emb(w_i)$, followed by a plus sign, and finally a vertical column of four red rounded rectangles representing the absolute positional encoding $Emb_{pos}(i)$.

$$x_i = Emb(w_i) + Emb_{pos}(i)$$

Relative Position Encodings (RPE)

- При реализации генерируется матрица относительных позиций и из нее находятся эмбеддинги
- Для возможности адаптации модели к длинным текстам можно ограничить максимальное расстояние между токенами

$$Attn_i = \text{softmax}\left(\frac{Q_i^T(K^T + R_i^K)}{\sqrt{d}}\right)(V + R_i^V)$$

$$P = \begin{pmatrix} 0 & 1 & 2 & 2 & 2 \\ -1 & 0 & 1 & 2 & 2 \\ -2 & -1 & 0 & 1 & 2 \\ -2 & -2 & -1 & 0 & 1 \\ -2 & -2 & -2 & -1 & 0 \end{pmatrix}$$

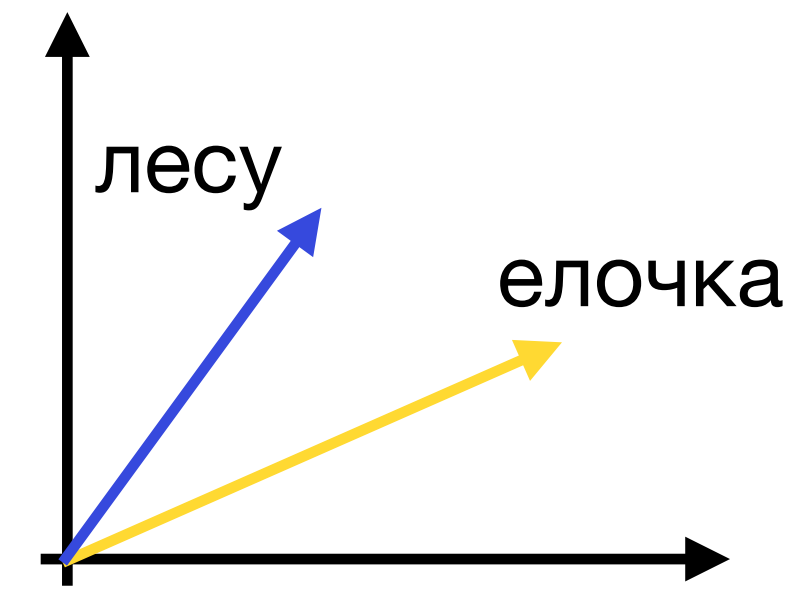
$$R^K = \text{Emb}_K(P) \in \mathbb{R}^{[n \times n \times d]}$$

$$R^V = \text{Emb}_V(P) \in \mathbb{R}^{[n \times n \times d]}$$

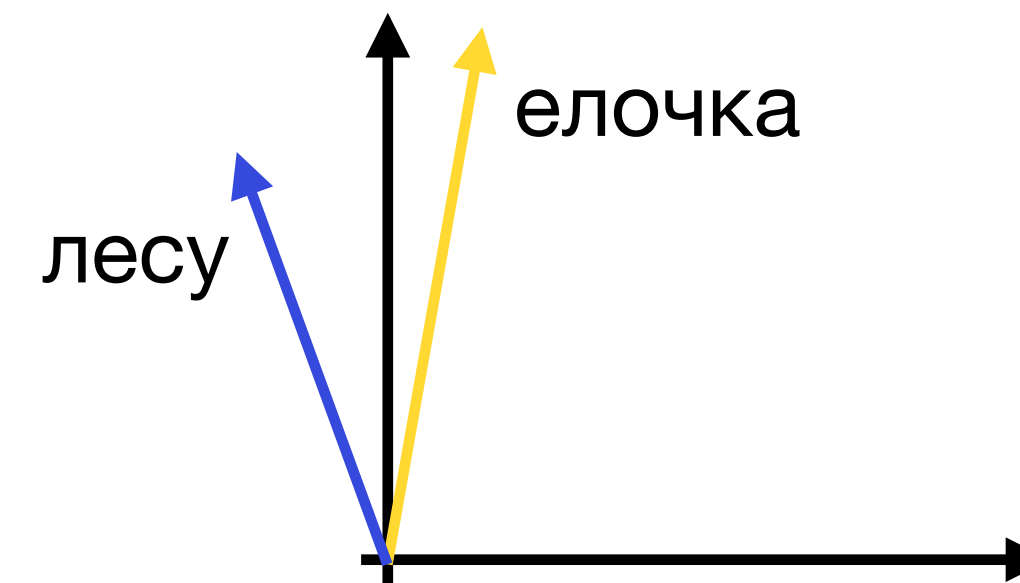
Rotary Position Embeddings (RoPE)

- Векторы q и k поворачиваются на угол $i\theta$, где i – позиция в тексте
- Таким образом, относительное расстояние не меняется при изменении позиции
- Векторы для похожих позиций будут поворачиваться на похожий угол, далекие векторы – на разные углы

в **лесу** родилась **елочка**



в нашем зимнем **лесу** родилась **елочка**



Математическая запись RoPE

Операция поворота вектора производится с помощью домножения на матрицу поворота

$$Q_i^r = \begin{pmatrix} \cos i\theta & -\sin i\theta \\ \sin i\theta & \cos i\theta \end{pmatrix} Q_i \quad K_j^r = \begin{pmatrix} \cos j\theta & -\sin j\theta \\ \sin j\theta & \cos j\theta \end{pmatrix} K_j$$

Внимание считается так же, как и раньше, но с повернутыми матрицами Q и K

$$Attn = softmax\left(\frac{Q^r K^{rT}}{\sqrt{d}}\right) V$$

Многомерный случай RoPE

- Матрица поворота заменяется на блочно-диагональную матрицу
- Каждый блок – матрица поворота с определенным углом

$$R_j^d = \begin{pmatrix} \cos j\theta_1 & -\sin j\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin j\theta_1 & \cos j\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos j\theta_2 & -\sin j\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin j\theta_2 & \cos j\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos j\theta_{d/2} & -\sin j\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin j\theta_{d/2} & \cos j\theta_{d/2} \end{pmatrix},$$

$$Q_i^{rT} K_j^r = (R_i^d Q_i)^T (R_j^d K_j) = Q_i^T R_i^{dT} R_j^d K_j = Q_i^T R_{j-i}^d K_j$$

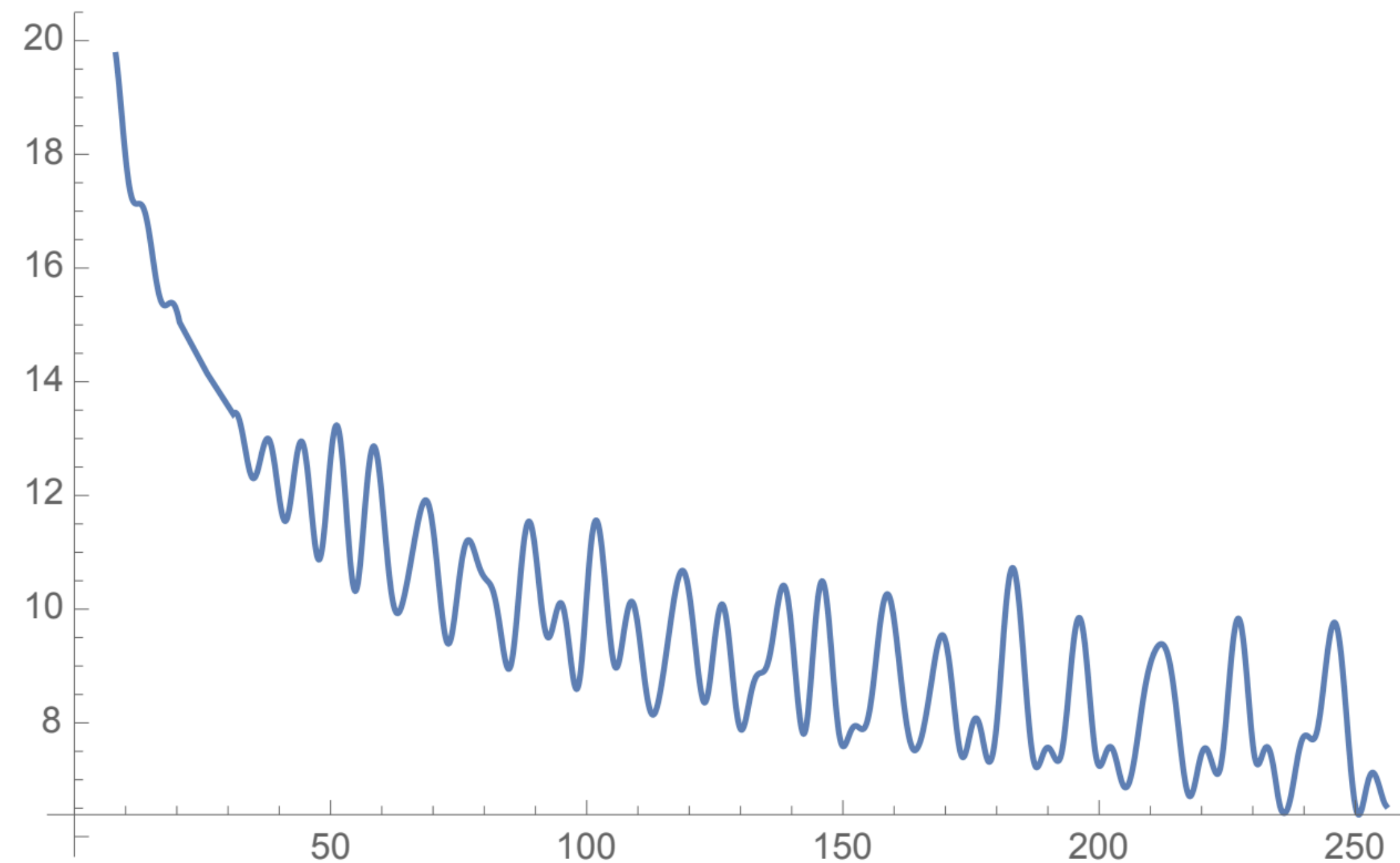
где $\theta_i = 10000^{-2(i-1)/d}$. Значение 10000 выбрано эмпирически.

Каждый угол задает свою частоту волны, поэтому часть координат хранит локальную информацию о позиции, а часть – глобальную

Обоснование RoPE

Благодаря выбранному углу, чем больше расстояние между токенами, тем меньше значение внимания между ними

Верхняя оценка на значение внимания



относительное расстояние

Эффективность подсчета RoPE

Пользуясь разреженностью матрицы R_j^d , можно значительно упростить умножение на нее

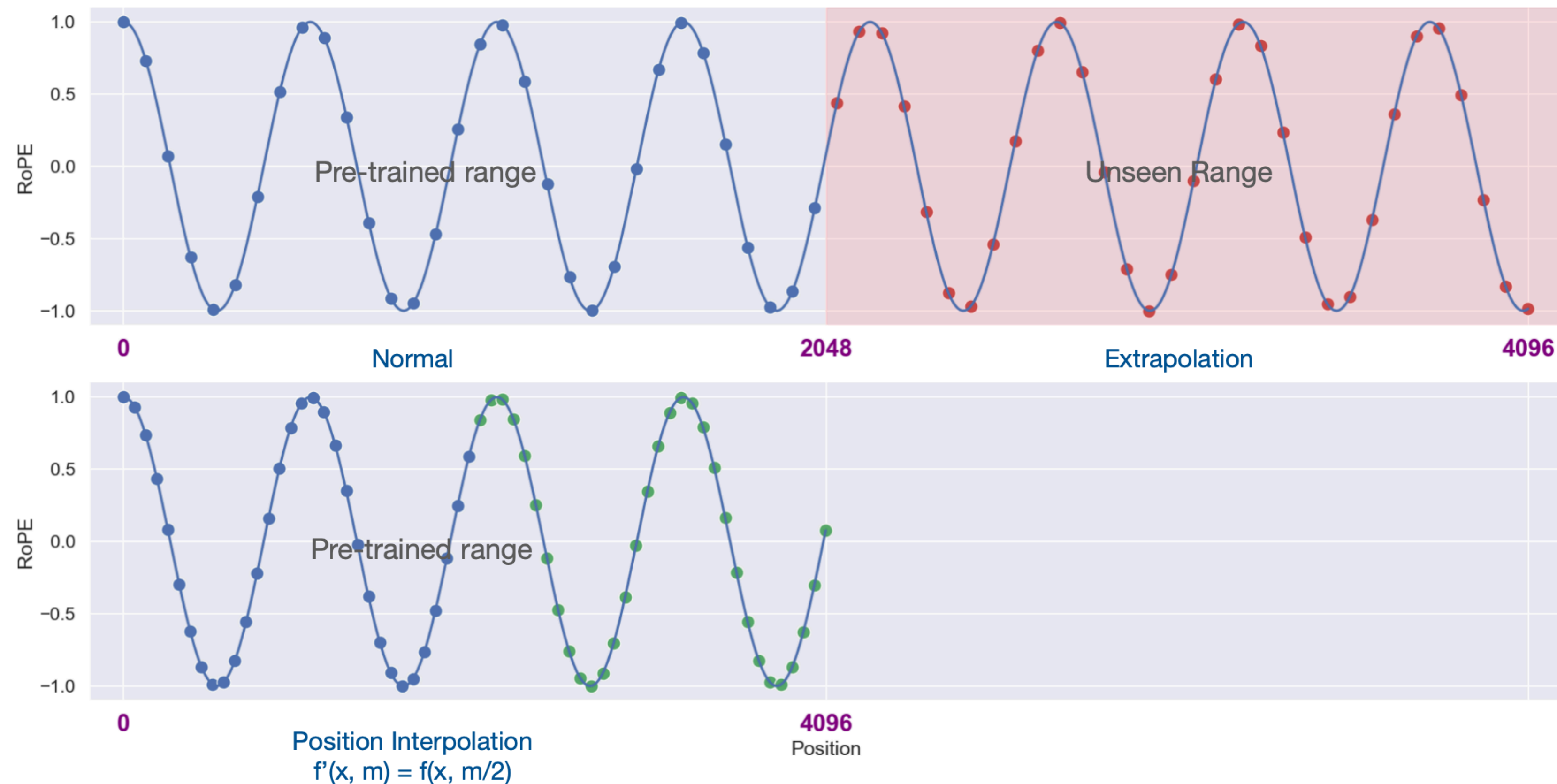
$$\begin{pmatrix} \cos j\theta & -\sin j\theta \\ \sin j\theta & \cos j\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \cos j\theta - x_2 \sin j\theta \\ x_2 \cos j\theta + x_1 \sin j\theta \end{pmatrix}$$

$$R_j^d x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos j\theta_1 \\ \cos j\theta_1 \\ \cos j\theta_2 \\ \cos j\theta_2 \\ \vdots \\ \cos j\theta_{d/2} \\ \cos j\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin j\theta_1 \\ \sin j\theta_1 \\ \sin j\theta_2 \\ \sin j\theta_2 \\ \vdots \\ \sin j\theta_{d/2} \\ \sin j\theta_{d/2} \end{pmatrix}$$

Таким образом, умножение занимает $O(d)$ операций вместо $O(d^2)$.

Интерполяция RoPE

Экстраполяция не работает для RoPE, зато интерполяция – отлично

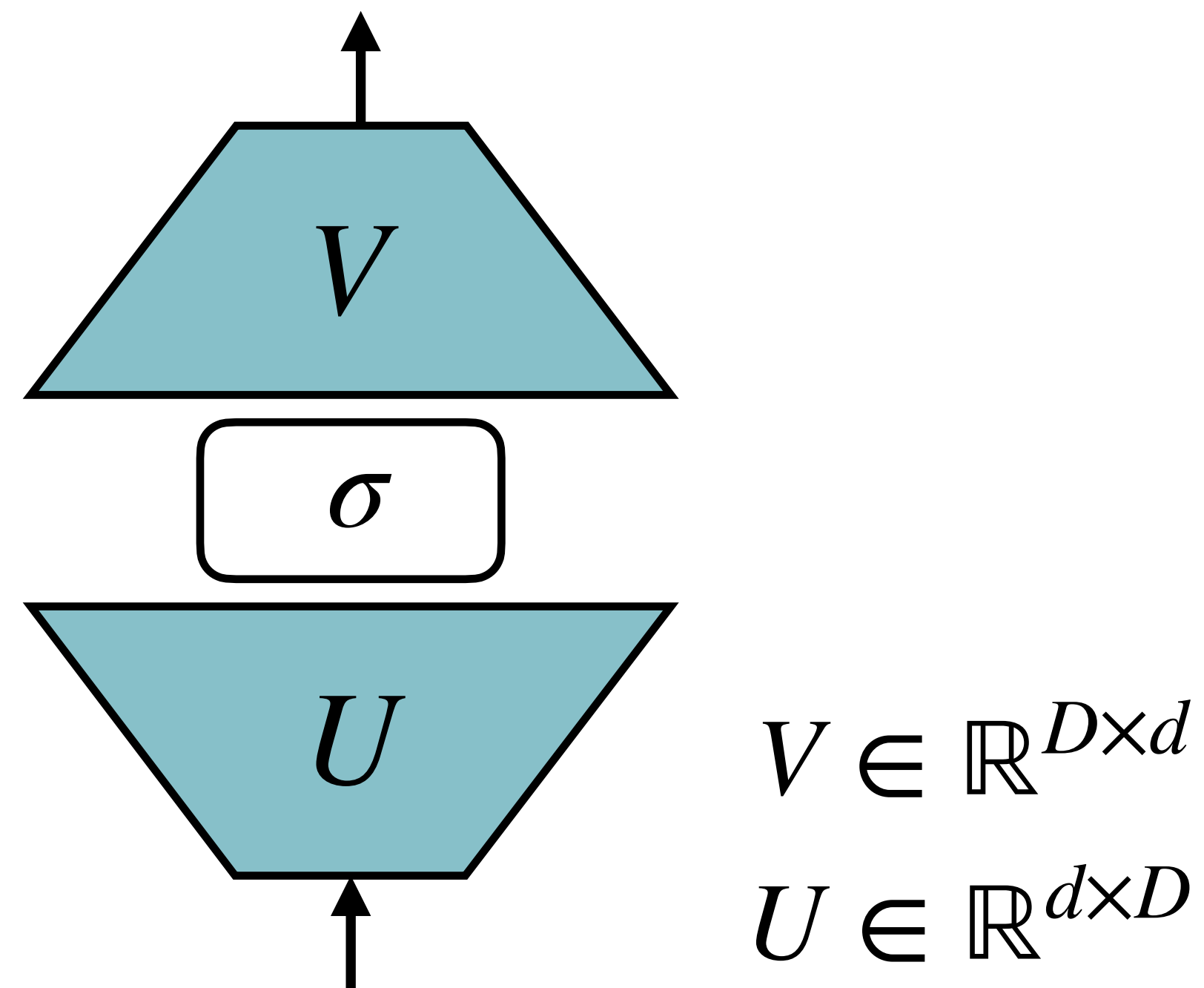


Feed forward network

Feed forward network

- FFN состоит из двух линейных слоев с нелинейностью между ними
- Первый слой повышает размеренность, а второй понижает обратно

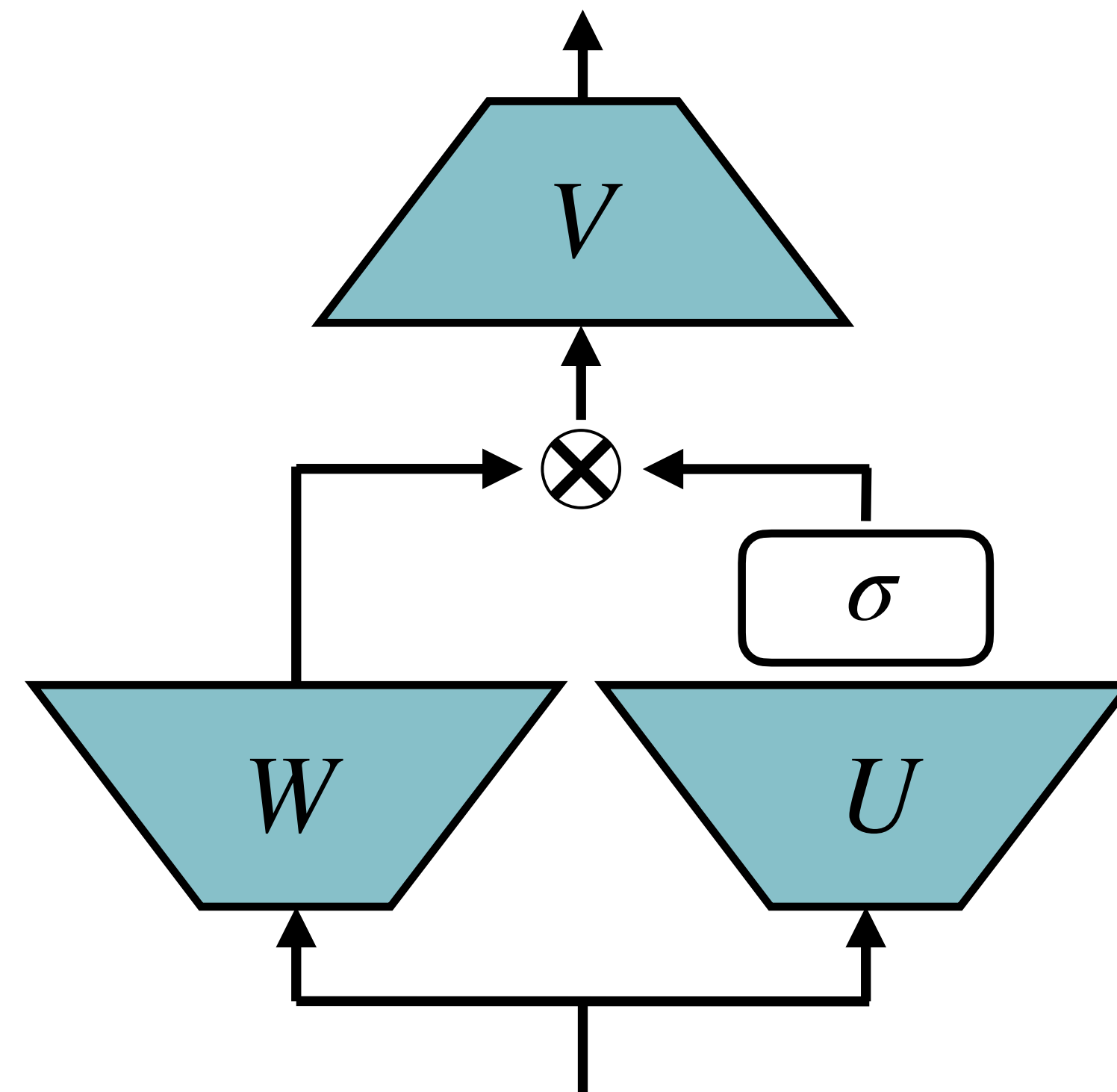
$$FFN(x) = \sigma(xU + b_u)V + b_v$$



Gated Linear Unit (GLU)

- GLU добавляет дополнительный линейный слой W , выходы которого умножаются на выходы U после активации
- W играет роль фильтра, отсеивающего ненужные компоненты

$$FFN_{GLU}(x) = (\sigma(xU + b_u) \otimes (xW + b_w))V + b_v$$



$$V \in \mathbb{R}^{D \times d}$$

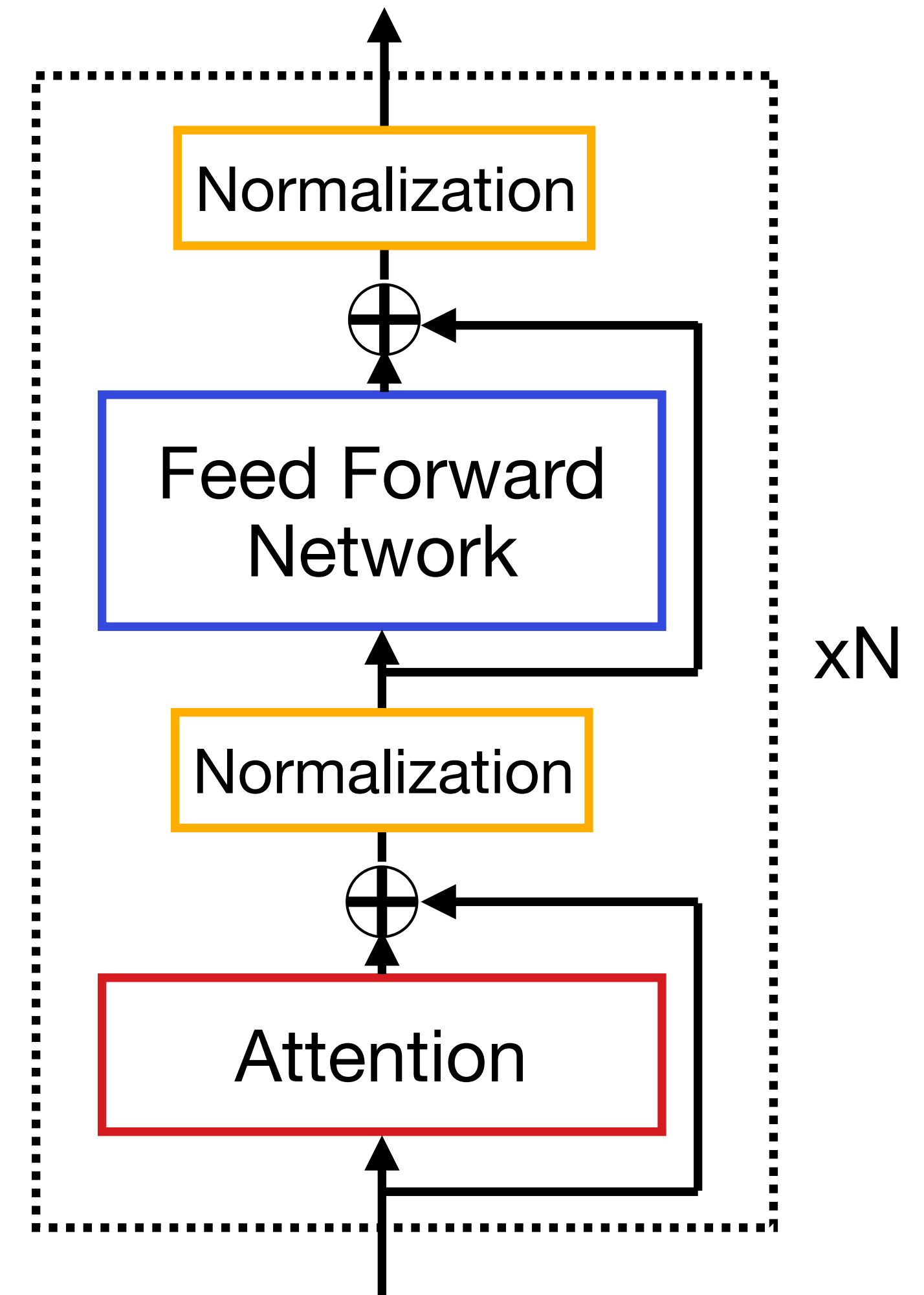
$$U \in \mathbb{R}^{d \times D}$$

$$W \in \mathbb{R}^{d \times D}$$

Нормализация

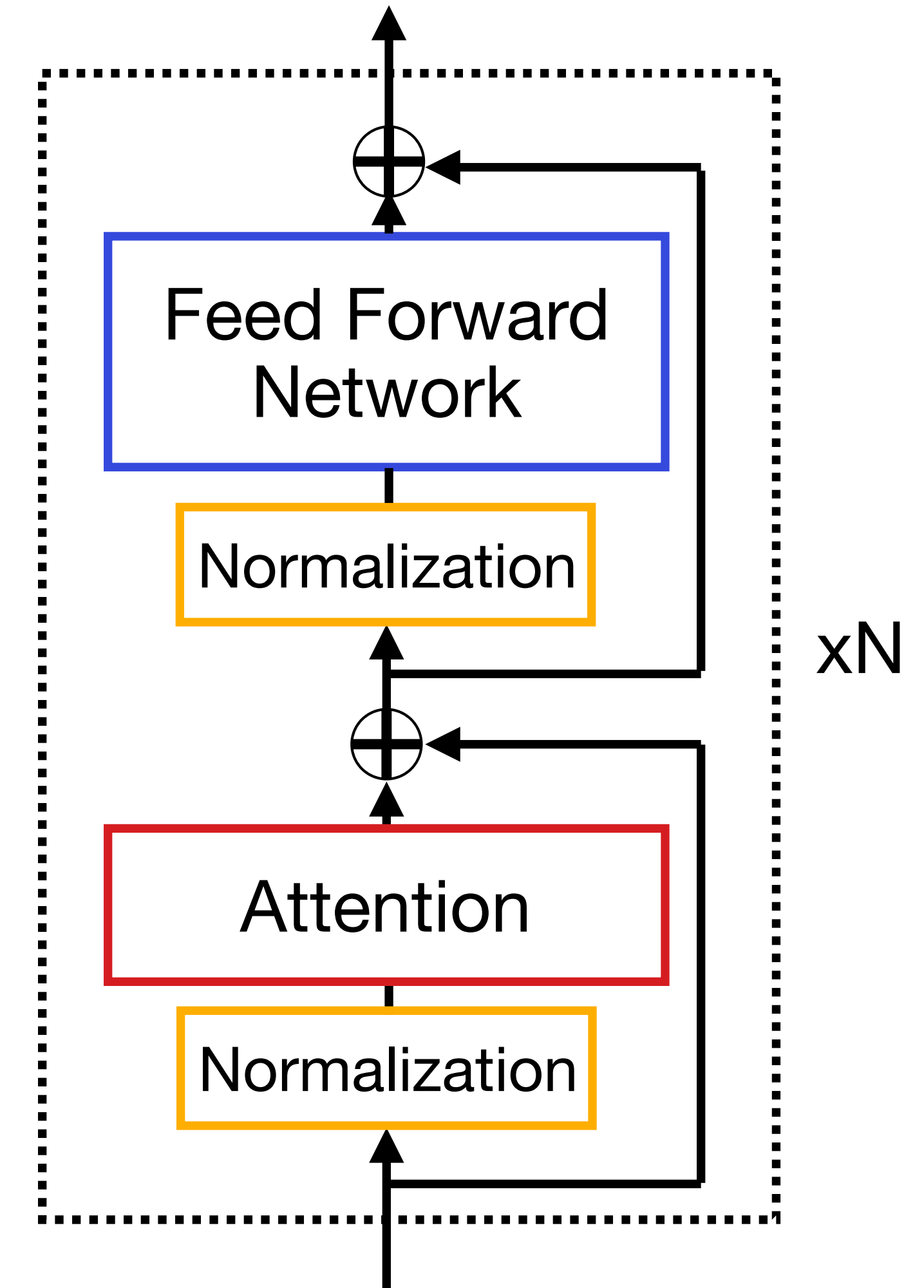
Нормализация

- Нормализация стабилизирует обучение, не позволяя выходам слоев расходиться
- Однако из-за нормализации градиенты начинают затухать сразу после начала обучения, и модель вообще не учится
- Именно поэтому используется warm-up



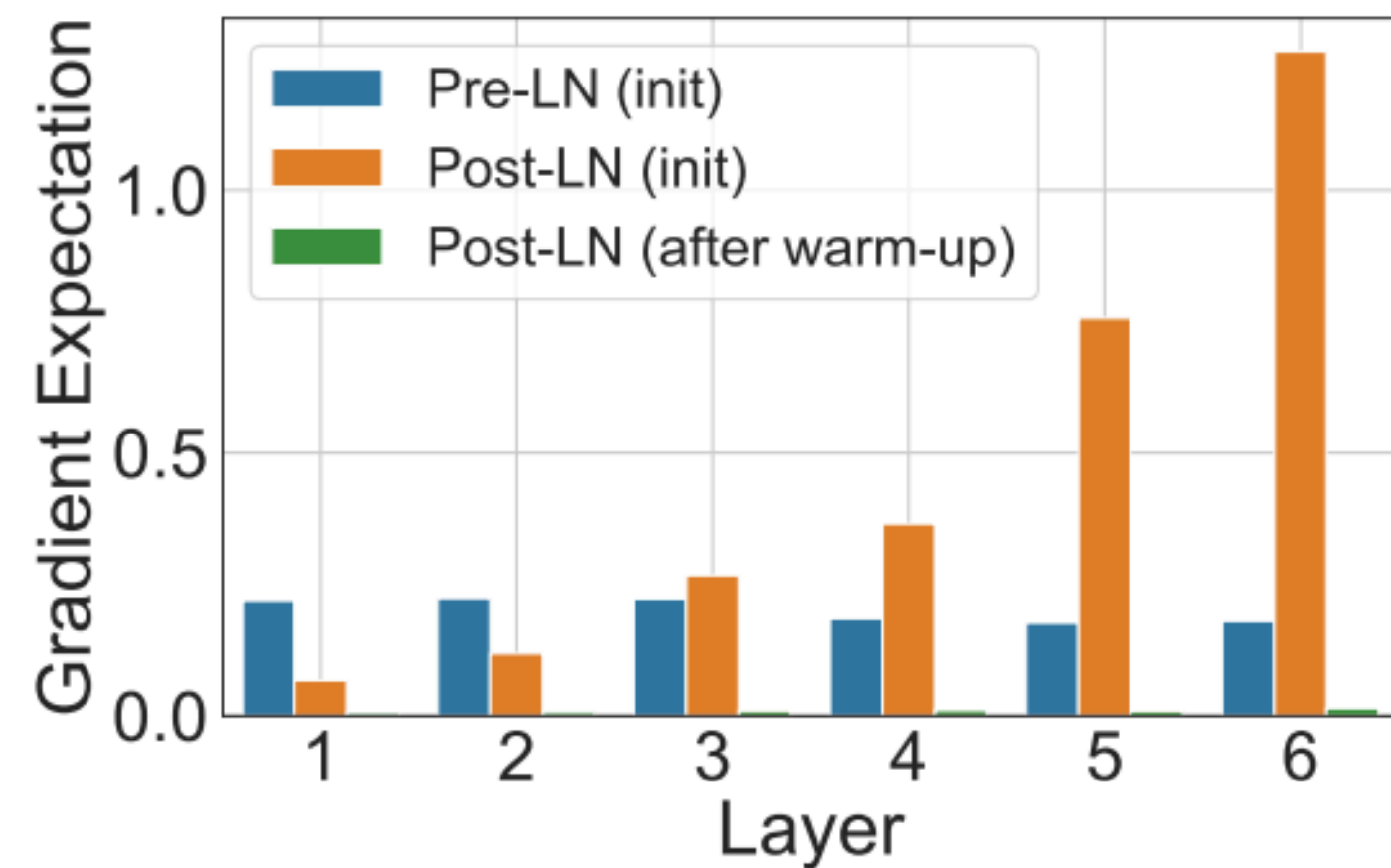
Pre-normalization

- Можно нормировать не все тензоры, а только входы для слоев внимания и FFN
- Такой подход называется **Pre-LN Transformer**
- Стандартный подход – **Post-LN**
- Градиенты теперь ведут себя стабильнее и Трансформер можно учить без warm-up

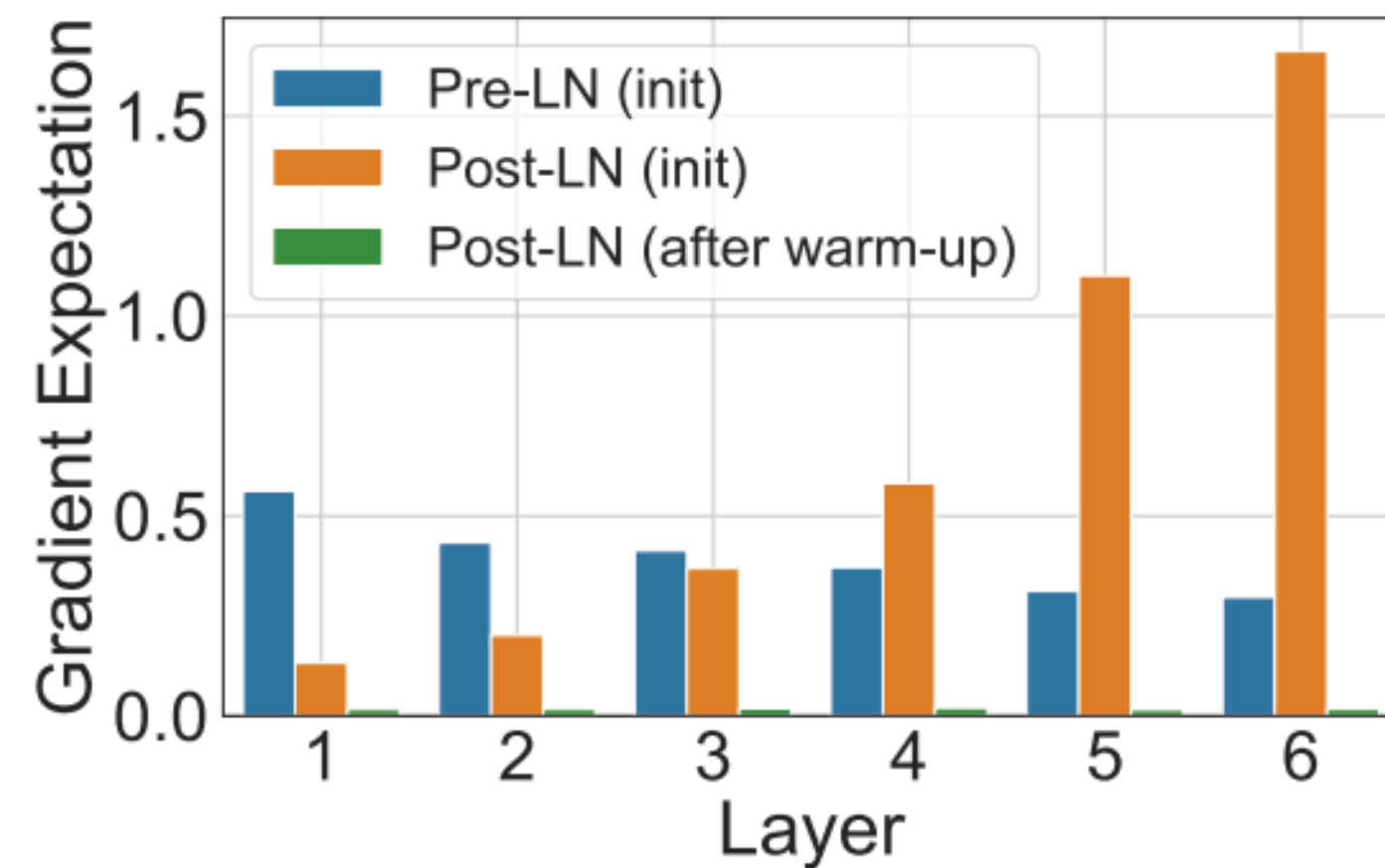


Поведение градиентов

- Градиенты у **Post-LN** подхода без warm-up увеличиваются с увеличением номера слоя
- У **Pre-LN** такого не происходит
- **Pre-LN** часто используется для обучения глубоких Трансформеров: Mistral, BLOOM, Falcon, Qwen2 и т.д.
- Для небольших Трансформеров качество **Pre-LN** обычно хуже



(a) W^1 in the FFN sub-layers



(b) W^2 in the FFN sub-layers

Layer Normalization

По умолчанию в Трансформерах используется Layer Normalization

$$y = \frac{x - \mathbb{E}(x)}{\sqrt{\text{Var}(x) + \epsilon}} \cdot \gamma + \beta$$

То есть считается выборочное среднее и дисперсия

$$\mathbb{E}(x) = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{Var}(x) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mathbb{E}(x))^2$$

Root Mean Square (RMS) Norm

По умолчанию в Трансформерах используется Layer Normalization

$$y = \frac{x - \mathbb{E}(x)}{\sqrt{\text{Var}(x) + \epsilon}} \odot \gamma + \beta$$

То есть считается выборочное среднее и дисперсия

$$\mathbb{E}(x) = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{Var}(x) = \frac{1}{n} \sum_{i=1}^n (x_i - \mathbb{E}(x))^2$$

Оказывается, можно считать среднее нулевым и не использовать его.

$$y = \frac{x}{\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2 + \epsilon}} \odot \gamma \quad \gamma \in \mathbb{R}^d$$

Root Mean Square (RMS) Norm

- RMSNorm часто работает даже лучше, чем Layer Norm
- RMSNorm считается быстрее на ~25%, так как не надо оценивать среднее по тензору
- Более того, для оценки дисперсии можно брать **не все** элементы тензора
- RMSNorm используется в современных моделях: Mistral, Llama, Qwen2 и т.д.

Model	Test14	Test17	Time
Baseline	21.7	23.4	399±3.40s
LayerNorm	22.6	23.6	665±32.5s
L2-Norm	20.7	22.0	482±19.7s
RMSNorm	22.4	23.7	501±11.8s (24.7%)
<i>p</i> RMSNorm	22.6	23.1	493±10.7s (25.9%)

Table 2: SacreBLEU score on newstest2014 (Test14) and newstest2017 (Test17) for RNNSearch using Tensorflow-version Nematus. “*Time*”: the time in second per 1k training steps. We set p to 6.25%. We highlight the best results in bold, and show the speedup of RMSNorm against Layer-Norm in bracket.

Baseline – без нормализации

Сэмплирование

Методы семплирования токенов

Модель выдает распределение вероятностей токенов



Как выбрать токен из этого распределения?

Жадное семплирование

Greedy Sampling

Можно всегда выбирать токен с максимальной вероятностью

$$y_t = \operatorname{argmax}_{y \in V} p(y | y_{<t})$$

Плюсы:

- Максимизируем вероятность текста

Минусы:

- Теряем разнообразие текста
 - Плохо, если генерация **безусловная**
 - Не страшно, если **условная** (seq2seq)
- 

Beam Search

Лучевой поиск

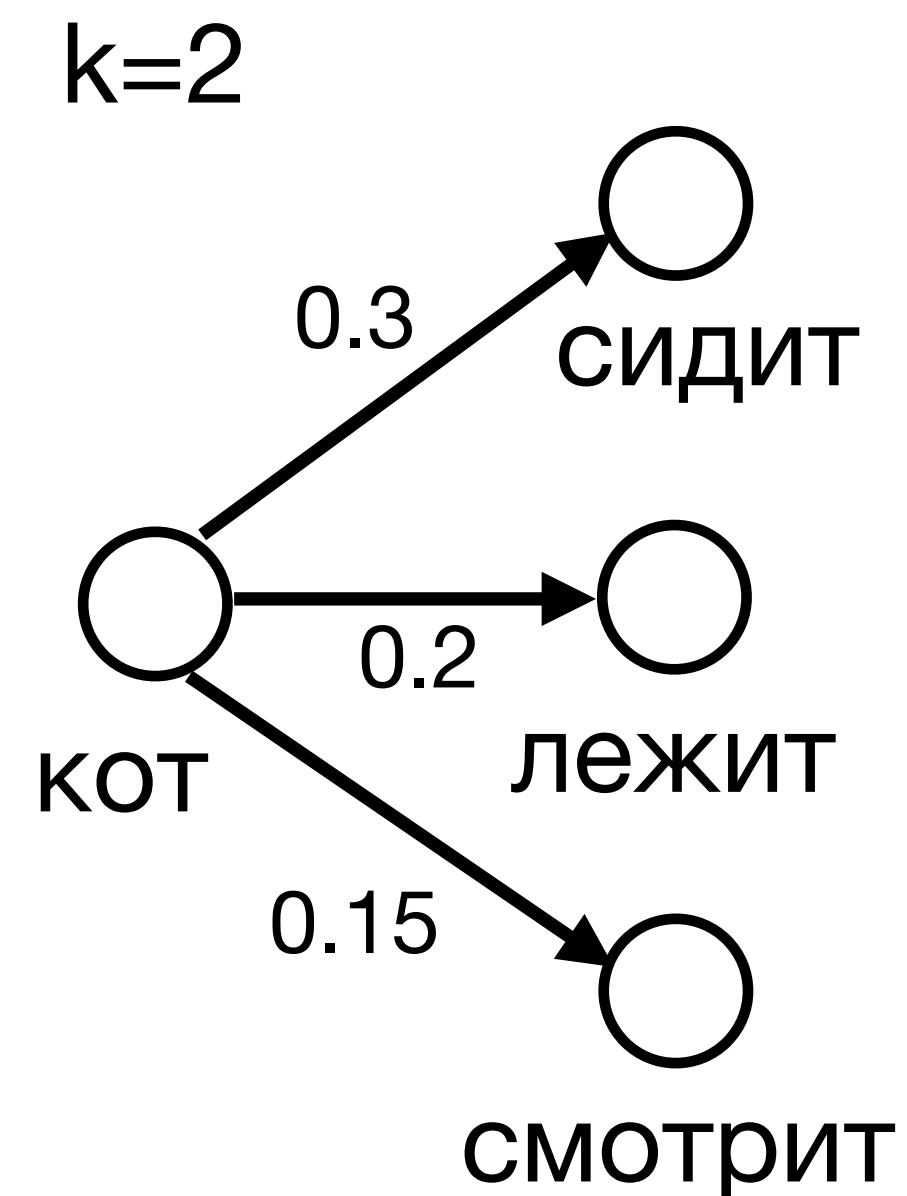
- Попробуем максимизировать вероятность текста еще больше
- При жадном семплировании мы **не учитываем** влияние токена на следующие за ним
- Будем строить предсказания на несколько токенов вперед, а затем выбирать токен с наибольшей совокупной вероятностью

Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

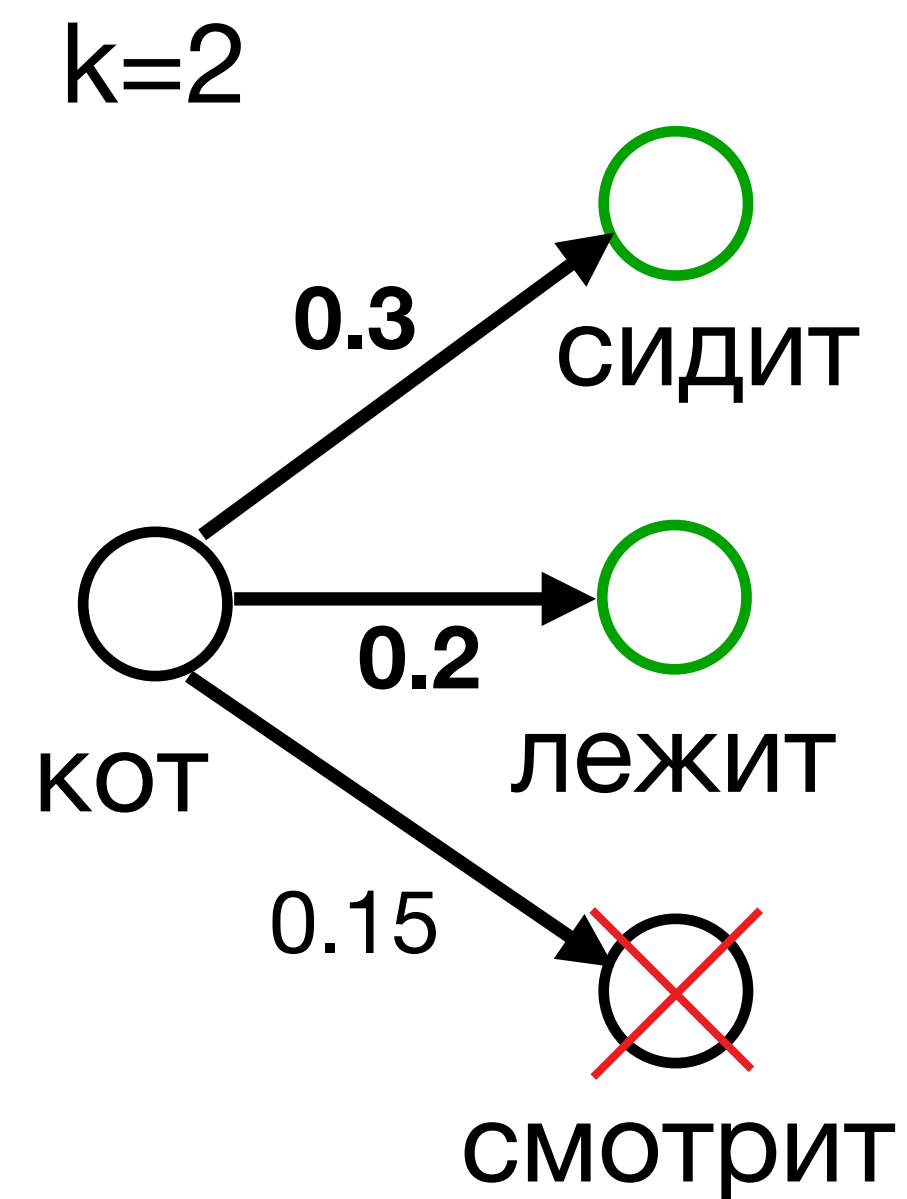


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

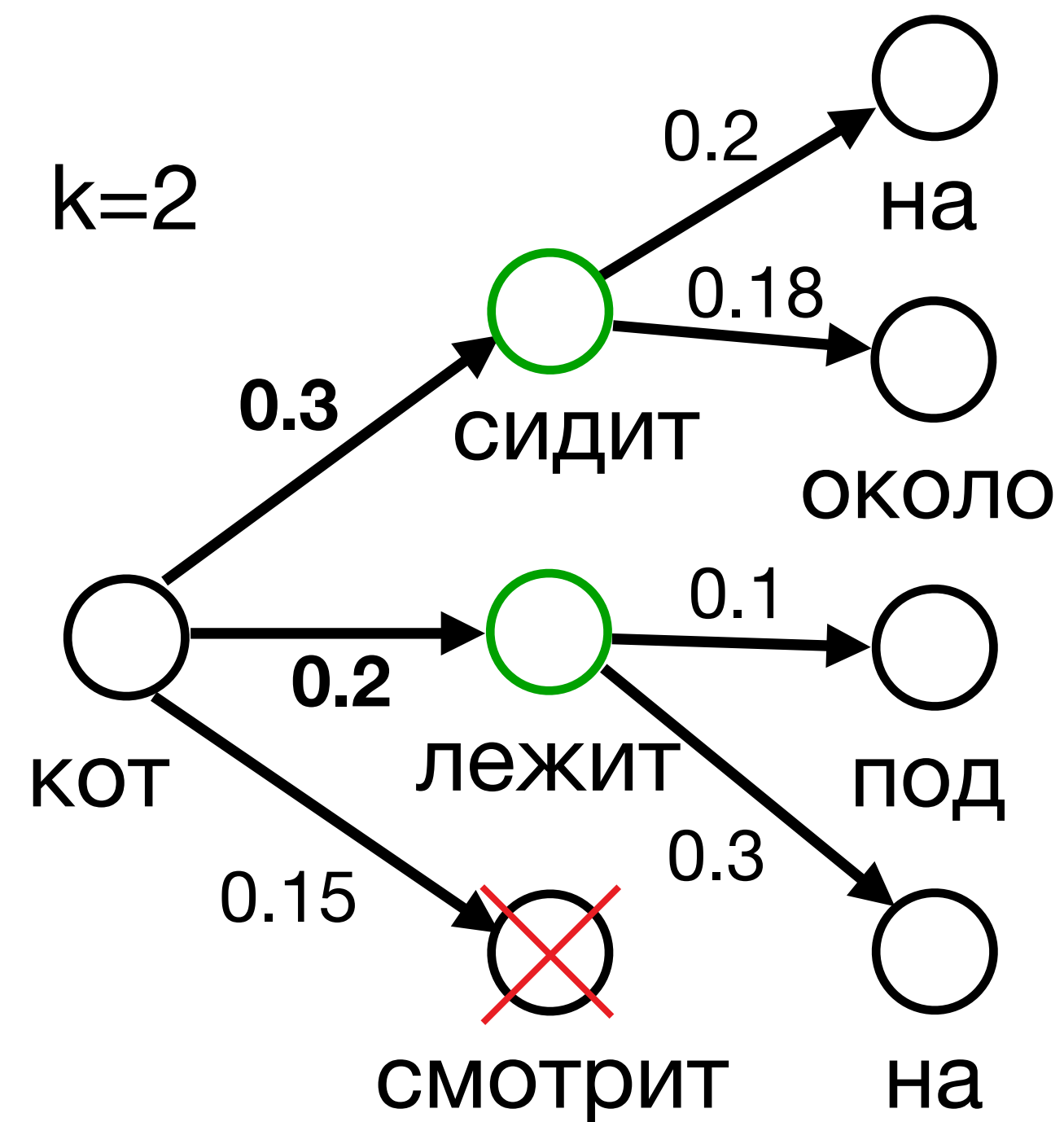


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

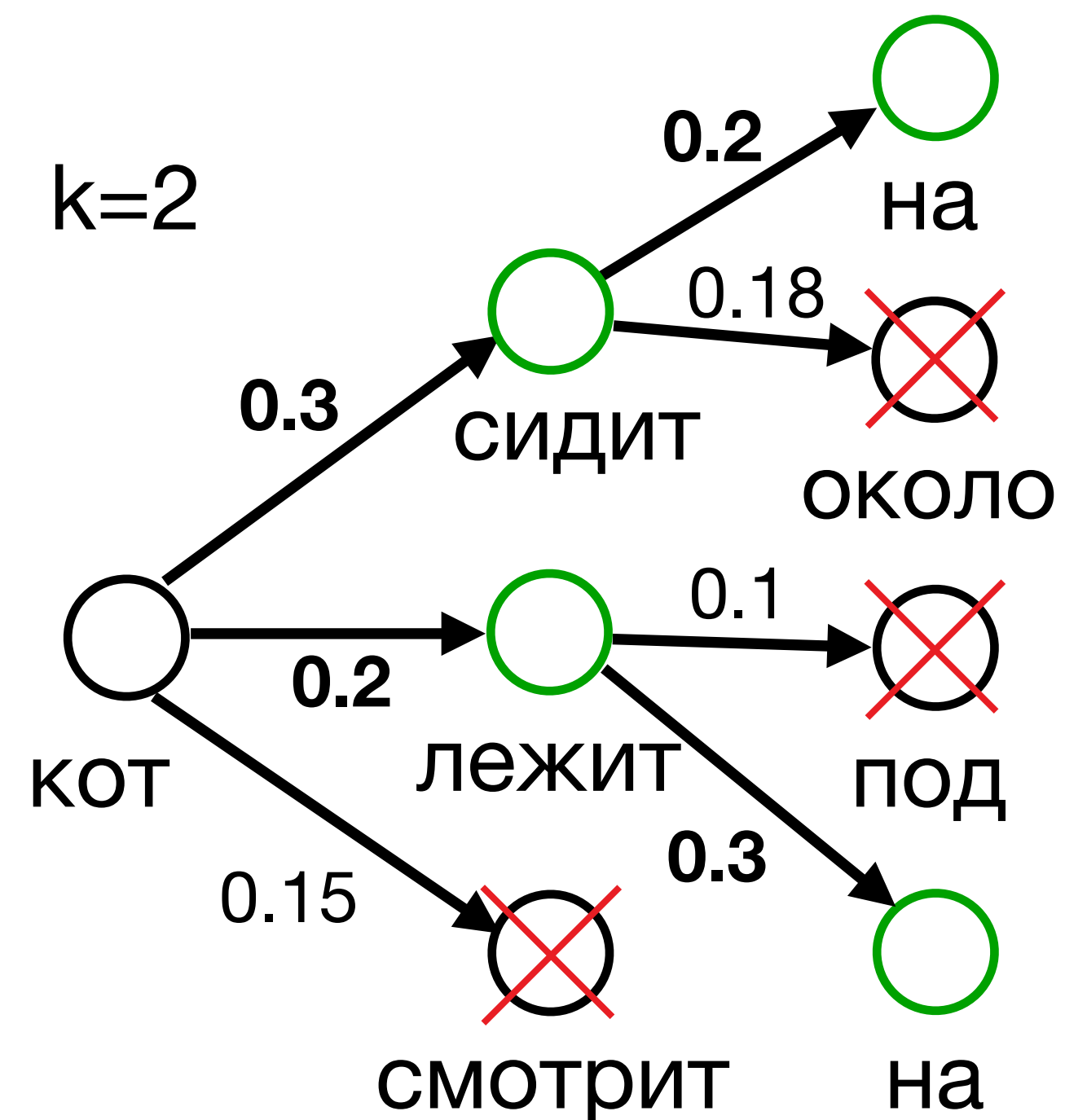


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

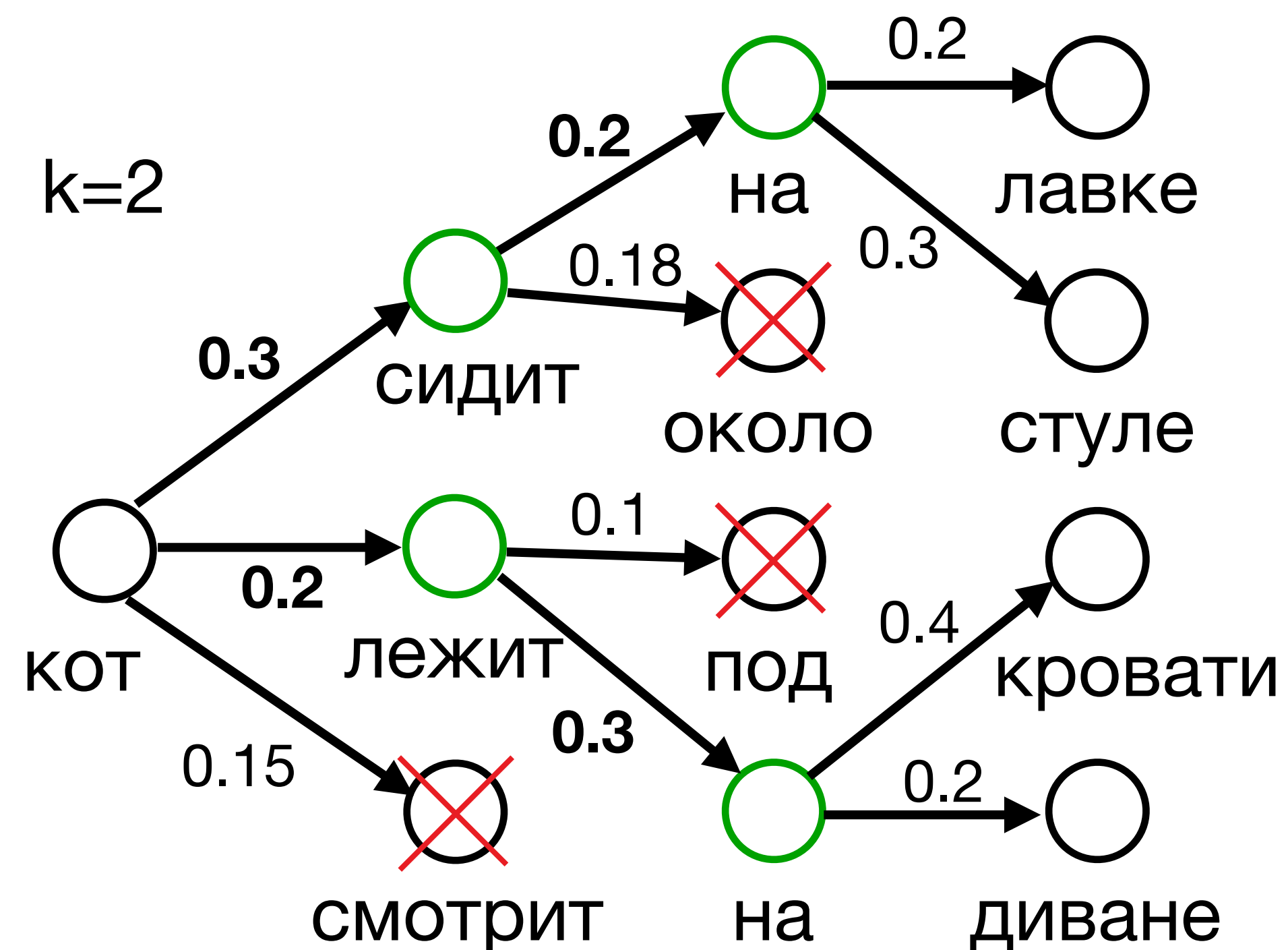


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

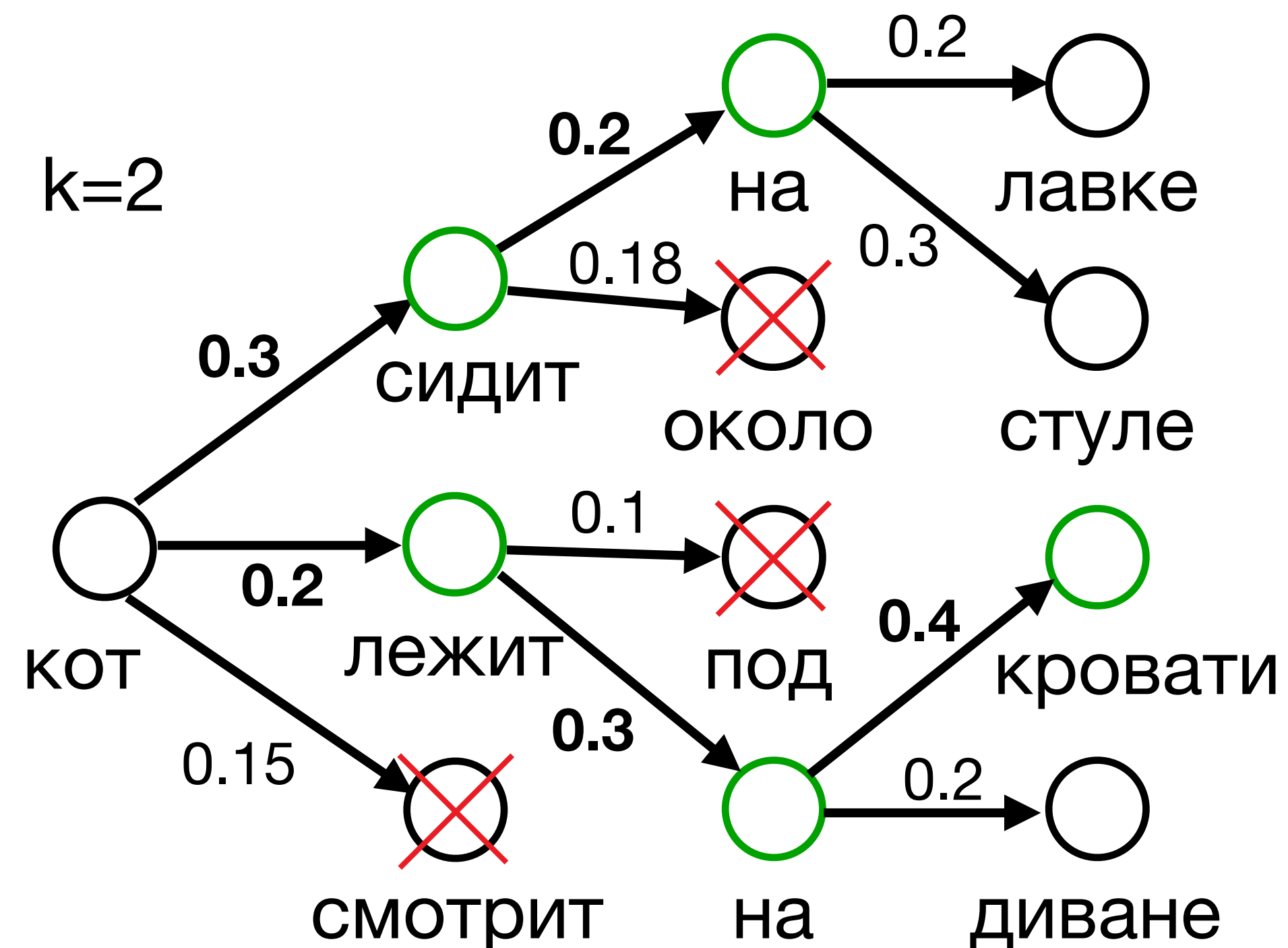


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных



Траектория с "лежит" имеет максимальную вероятность

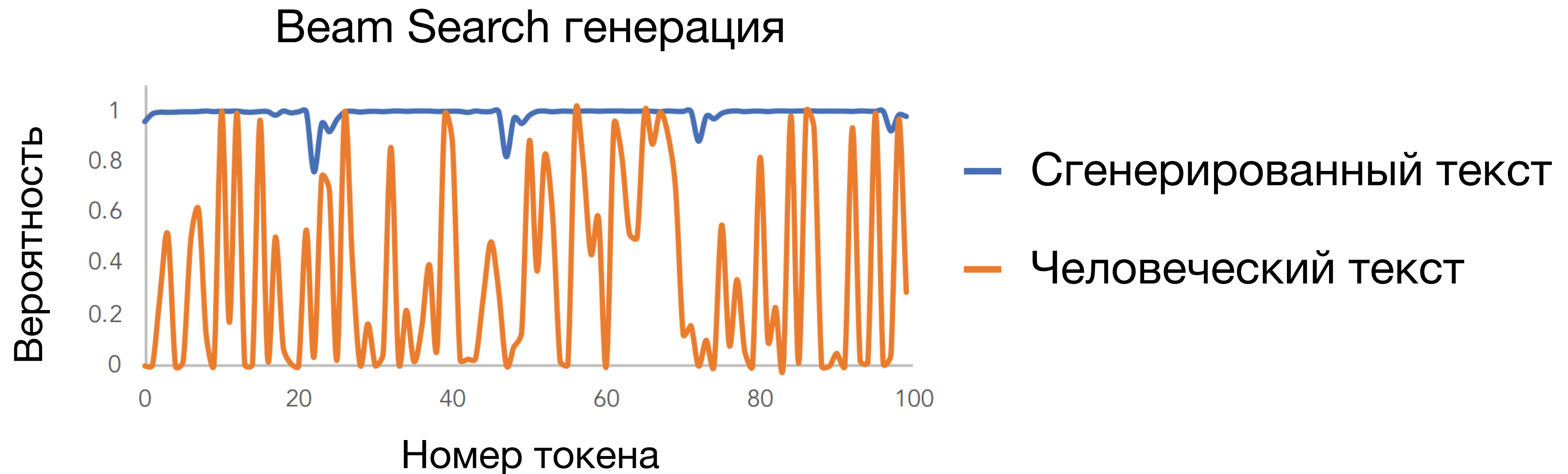
=> выбираем "лежит"

Beam Search vs Жадное семплирование

- Beam Search работает лучше для всех seq2seq задач
- Beam Search работает гораздо медленнее (зависит от k)
- Популярные значения k – 3 или 5

Безусловная генерация

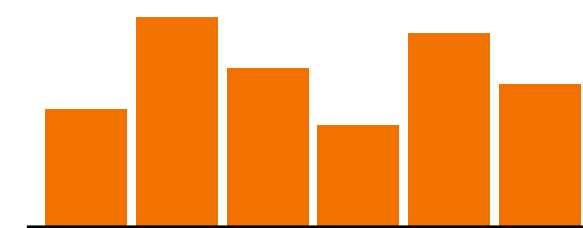
Оба метода уменьшают разнообразие безусловной генерации!



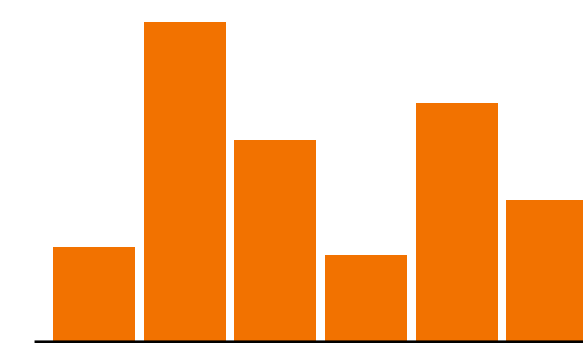
Семплирование с температурой

- При обычном семплировании с вероятностями вероятностная масса случайных токенов слишком велика
- Сделаем распределение более вырожденным, добавив температуру $\tau \in [0,1]$

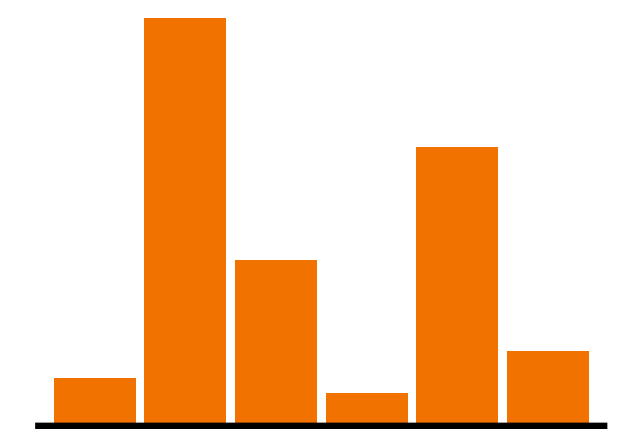
$$p_{\tau}(y | y_{<t}) = \frac{\exp \frac{p(y | y_{<t})}{\tau}}{\sum_{w \in V} \exp \frac{p(w | y_{<t})}{\tau}}$$



$p_2(y_t | y_{<t})$



$p_1(y_t | y_{<t})$

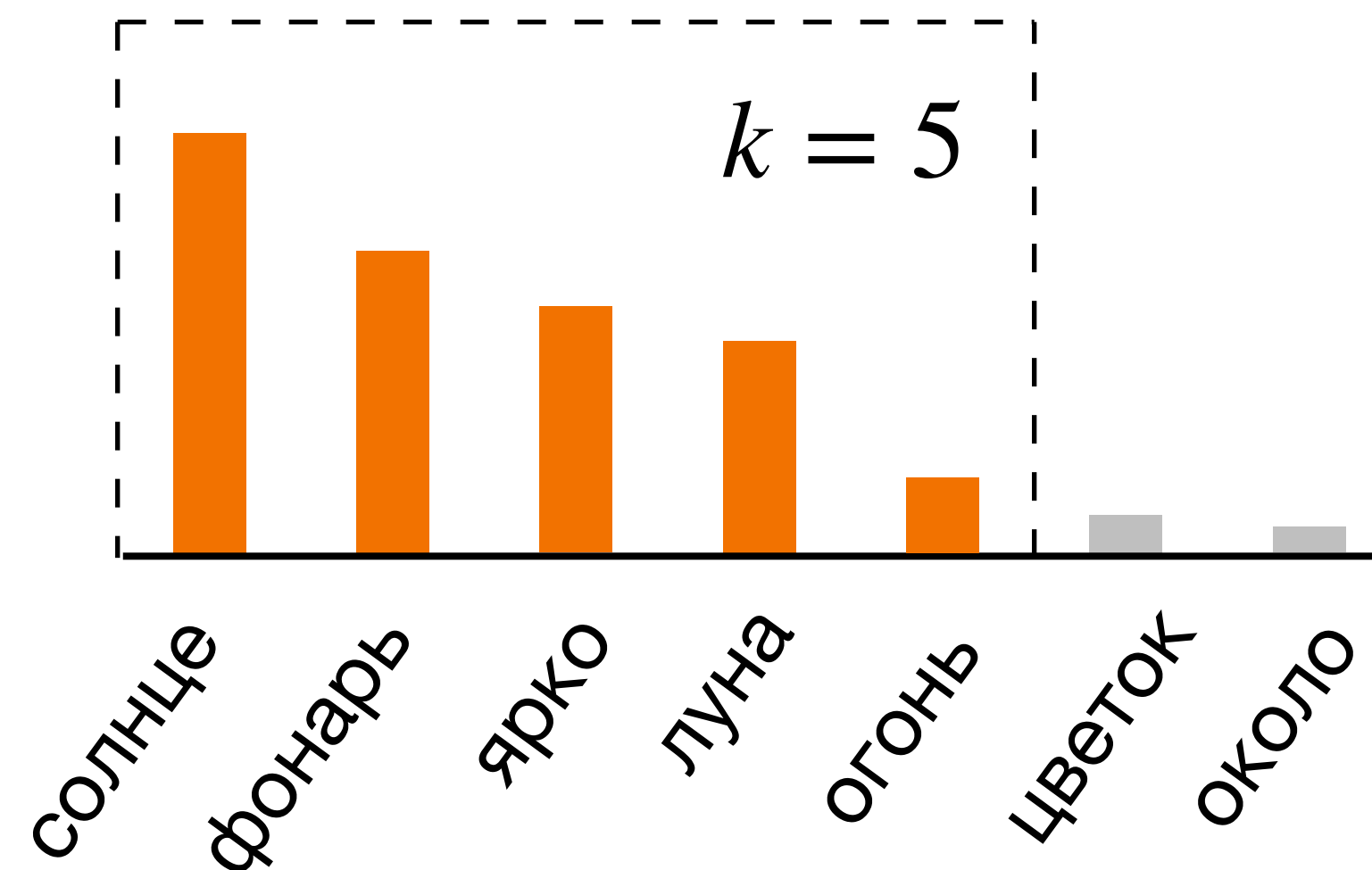


$p_{0.5}(y_t | y_{<t})$

Top-k семплирование

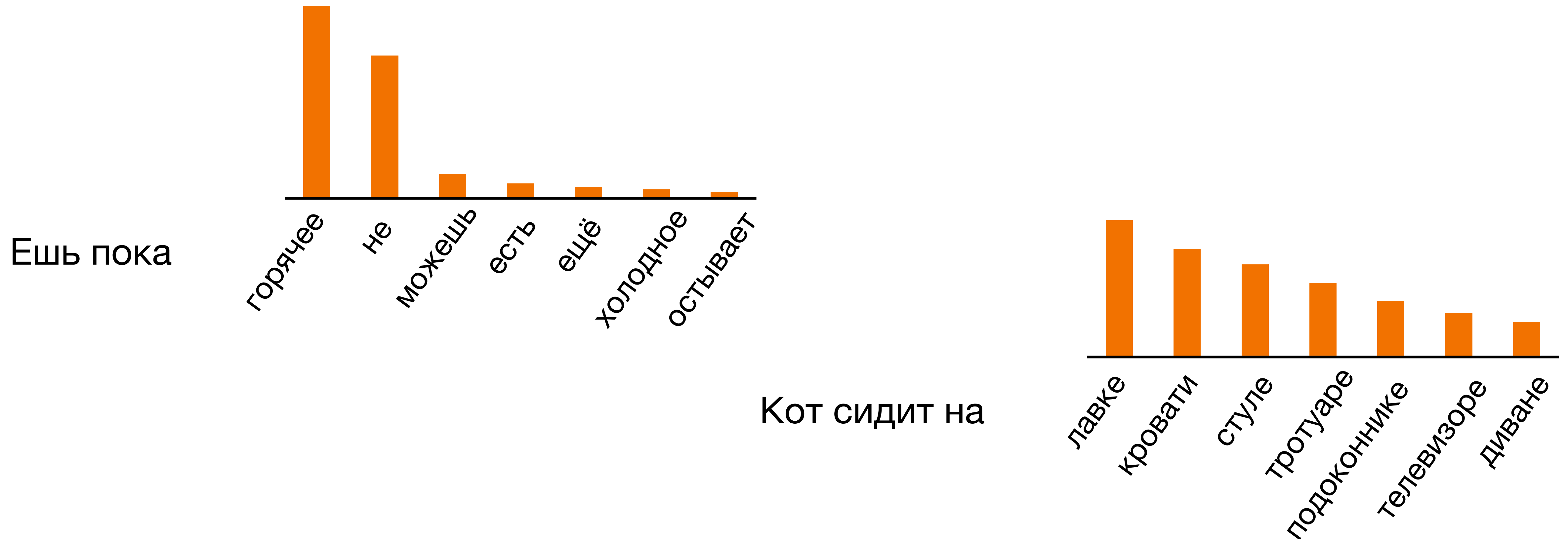
- Даже с температурой остается ненулевая генерация выдать случайный токен
- Оставим только k самых вероятных токенов и будем семплировать из них

На улице светит



Top-k семплирование

- Не во всех ситуациях самых вероятных токенов одинаковое число
- Из-за этого невозможно подобрать идеальное k



Тор-р семплирование

Nucleus sampling

- Будем выбирать из минимального числа токенов, суммарная вероятность которых больше p .

$$\sum_{w \in V^{(p)}} p(w | y_{<t}) \geq p$$

$$|V^{(p)}| \rightarrow \min$$

Популярное значение для p – 0.9 или 0.95